MET's Institute of Engineering

Bhujbal Knowledge City, Adgaon, Nashik.

Department of Computer Engineering

"HASHING"

Prepared By

Prof. Anand N. Gharu

(Assistant Professor)

Computer Dept.

CLASS : SE COMPUTER 2024

01September 2025

SUBJECT : DS (SEM-I)

UNIT : IV

Note: The material to prepare this presentation has been taken from internet and are generated only for students reference and not for commercial use.

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

SPPU Engineering & Technical UPDATES
WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

@SPPU_ENGINEERING_UPDATE

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

SYLLABUS

Hash Table- Concepts-hash table, hash function, basic operations, bucket, collision, probe, synonym, overflow, open hashing, closed hashing, perfect hash function, load density, full table, load factor, rehashing, issues in hashing, **hash functions-** properties of good hash function, division, multiplication, extraction, mid-square, folding and universal, **Collision resolution strategies-** open addressing and chaining, Hash table overflow- open addressing and chaining, extendible hashing, closed addressing and separate chaining.

Skip List- representation, searching and operations- insertion, removal

SYLLABUS

Hash Table : Concepts-hash table, hash function, basic operations, bucket, collision, probe, synonym, overflow, open hashing, closed hashing, perfect hash function, load density, full table, load factor, rehashing, properties of good hash function,

Collision resolution strategies- open addressing and chaining, Hash table overflow- open addressing and chaining, extendible hashing, closed addressing and separate chaining

Case study : Dictionary Application using Hash Tables, Description: Implement a dictionary where words and meanings are stored and retrieved using hashing with collision resolution

INTRODUCTION

UNIT-IV

HASHING

INTRODUCTION

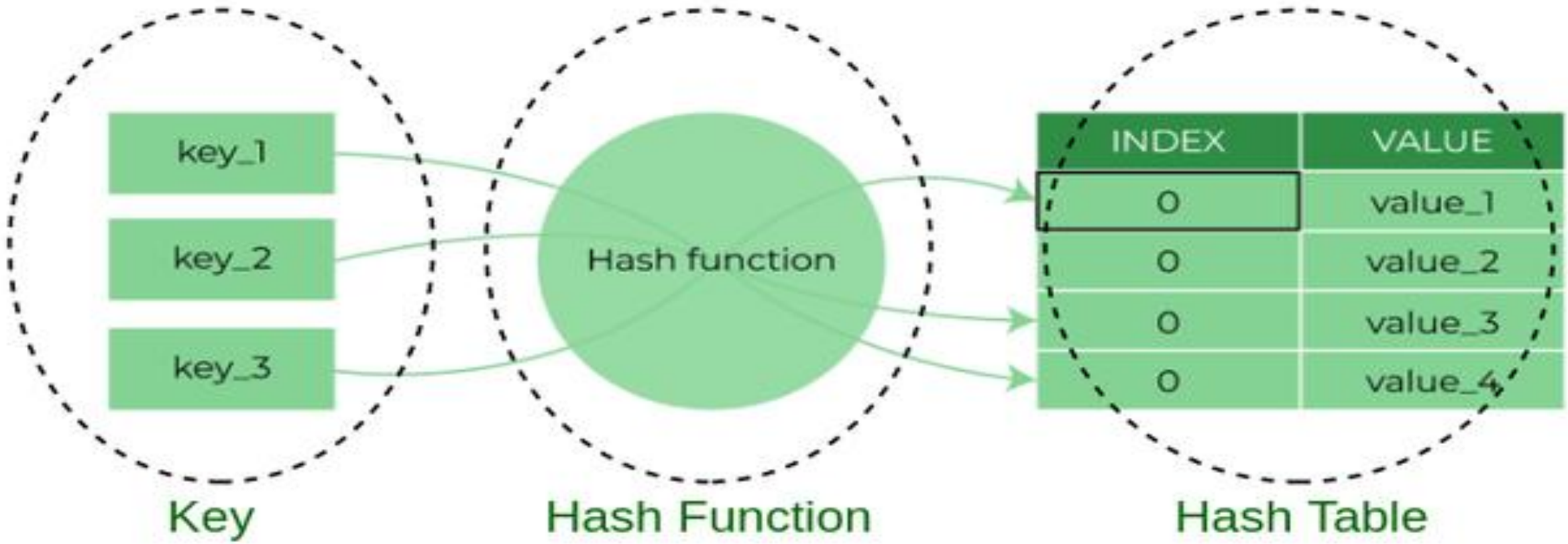
1. **Hashing** is finding an address where the data is to be stored as well as located using a key with the help of the algorithmic function.

2. **Hashing** is a method of directly computing the address of the record with the help of a key by using a suitable mathematical function called the hash function

3. **A hash table** is an array-based structure used to store <key, information> pairs

INTRODUCTION

4. Hash Table: A hash table is a data structure that stores records in an array, called a hash table. A Hash table can be used for quick insertion and searching.



Components of Hashing

INTRODUCTION

- The resulting address is used as the basis for storing and retrieving records and this address is called as home address of the record

- For array to store a record in a hash table, hash function is applied to the key of the record being stored, returning an index within the range of the hash table

- The item is then stored in the table of that index position

Hashing

- **Hashing** is a well-known technique to search any particular element among several elements.
- It minimizes the number of comparisons while performing the search.

Advantage-

1. Hashing is extremely efficient.
2. The time taken by it to perform the search does not depend upon the total number of elements.
3. It completes the search with constant time complexity O(1).

Hashing

- **Hashing Mechanism -**

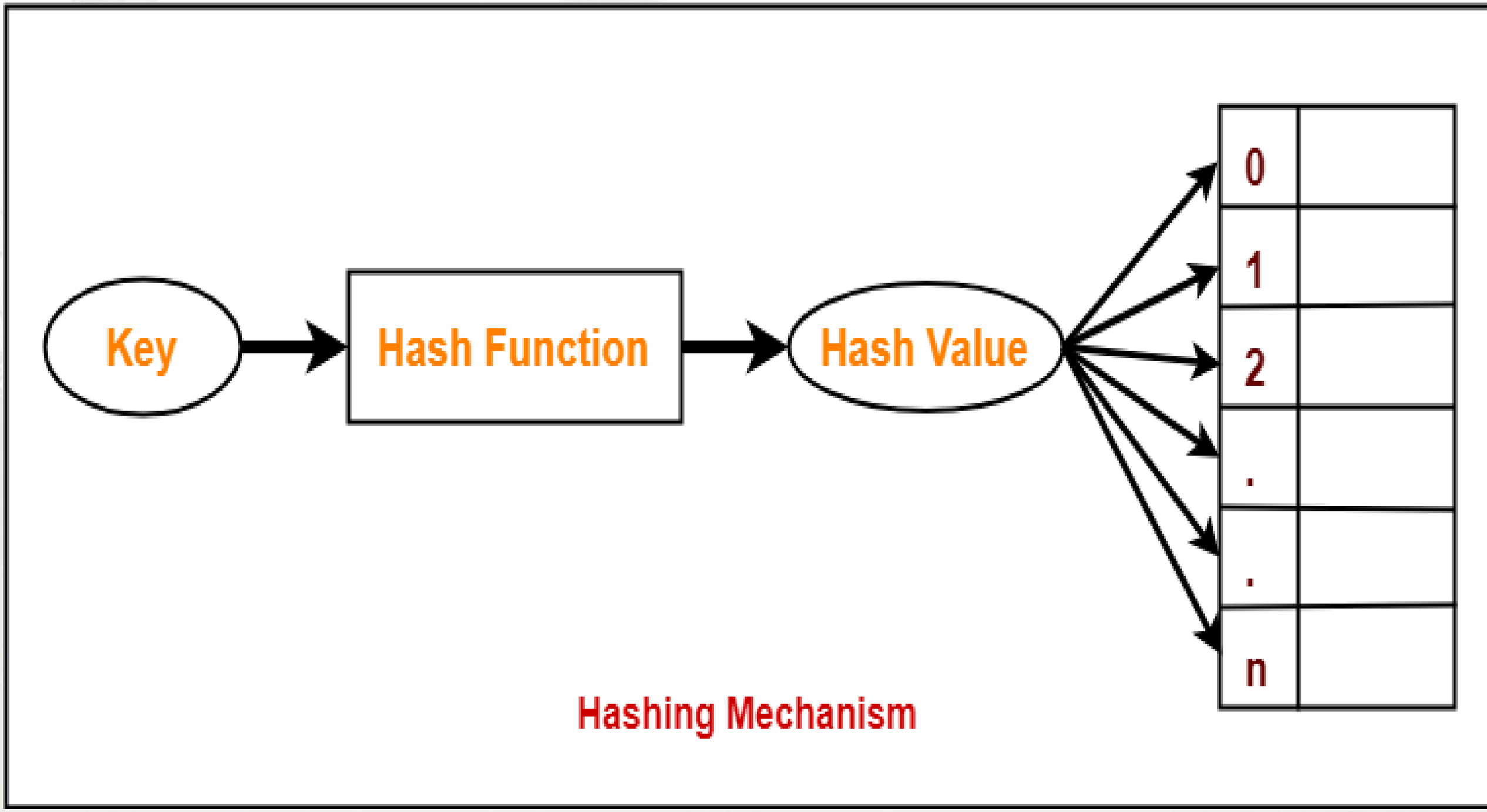
1. An array data structure called as Hash table is used to store the data items.
2. Based on the hash key value, data items are inserted into the hash table.

- **Hash Key Value -**

1. Hash key value is a special value that serves as an index for a data item.
2. It indicates where the data item should be stored in the hash table.
3. Hash key value is generated using a hash function.

Hashing

- **Hashing Mechanism :**



Hashing Mechanism

Hashing

- **Hash Function :**

1. Hash function takes the data item as an input and returns a small integer value as an output.
2. The small integer value is called as a hash value.
3. Hash value of the data item is then used as an index for storing it into the hash table.

Types of Hash Functions-

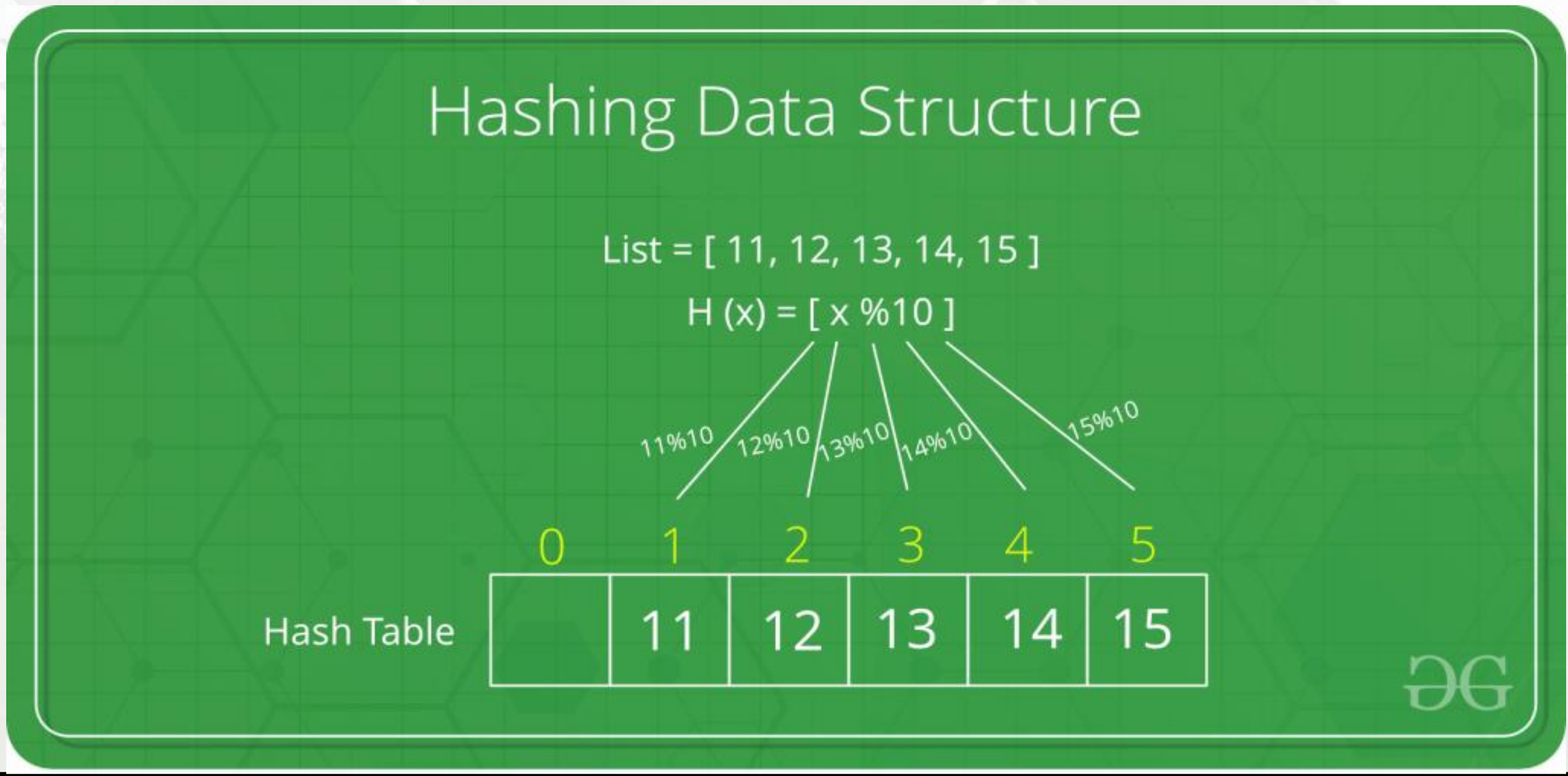
There are various types of hash functions available such as-

1. Mid Square Hash Function
2. Division Hash Function
3. Folding Hash Function etc

It depends on the user which hash function he wants to use.

HASH TABLE

- **A hash table** is a data structure that stores records in an array, called a hash table. A Hash table can be used for quick insertion and searching.



HASH TABLE

- **Hash table** is one of the most important data structures that uses a special function known as a **hash function** that maps a given value with a key to access the elements faster.

- **A Hash table** is a data structure that stores some information, and the information has basically two main components, **i.e., key and value**. The hash table can be implemented with the help of an **associative array**. The efficiency of mapping depends upon the efficiency of the hash function used for mapping.

ADVANTAGES OF HASH TABLE

Here, are pros/benefits of using hash tables:

1. Hash tables have high performance when looking up data, inserting, and deleting existing values.
2. The time complexity for hash tables is constant regardless of the number of items in the table.
3. They perform very well even when working with large datasets.

DISAVANTAGES OF HASH TABLE

Here, are cons of using hash tables:

1. You cannot use a null value as a key.
2. Collisions cannot be avoided when generating keys using. hash functions. Collisions occur when a key that is already in use is generated.
3. If the hashing function has many collisions, this can lead to performance decrease.

OPERATIONS OF HASH TABLE

Here, are the Operations supported by Hash tables:

1. **Insertion –** this Operation is used to add an element to the hash table
2. **Searching –** this Operation is used to search for elements in the hash table using the key
3. **Deleting –** this Operation is used to delete elements from the hash table

APPLICATIONS OF HASH TABLE

Real-world Applications

In the real-world, hash tables are used to store data for

1. Databases
2. Associative arrays
3. Sets
4. Memory cache

CHARACTRISTICS OF HASH FUNCTION

a) **Easy to compute :** It should be easy to compute and must not become an algorithm in itself.

b) Uniform distribution : It should provide a uniform distribution across the hash table and should not result in clustering.

c) **Less collisions** : Collisions occur when pairs of elements are mapped to the same hash value. These should be avoided.

d) Be easy and quick to compute.

e) Use all the information provided in the key

PROPERTIES OF HASH FUNCTION

1) Hash function should be simple to compute.

2) Number of collision should be less

2) The hash function uses all the input data.

3) The hash function "uniformly" distributes the data across the entire set of possible hash values.

4) The hash function generates very different hash values for similar strings.

HASH FUNCTION

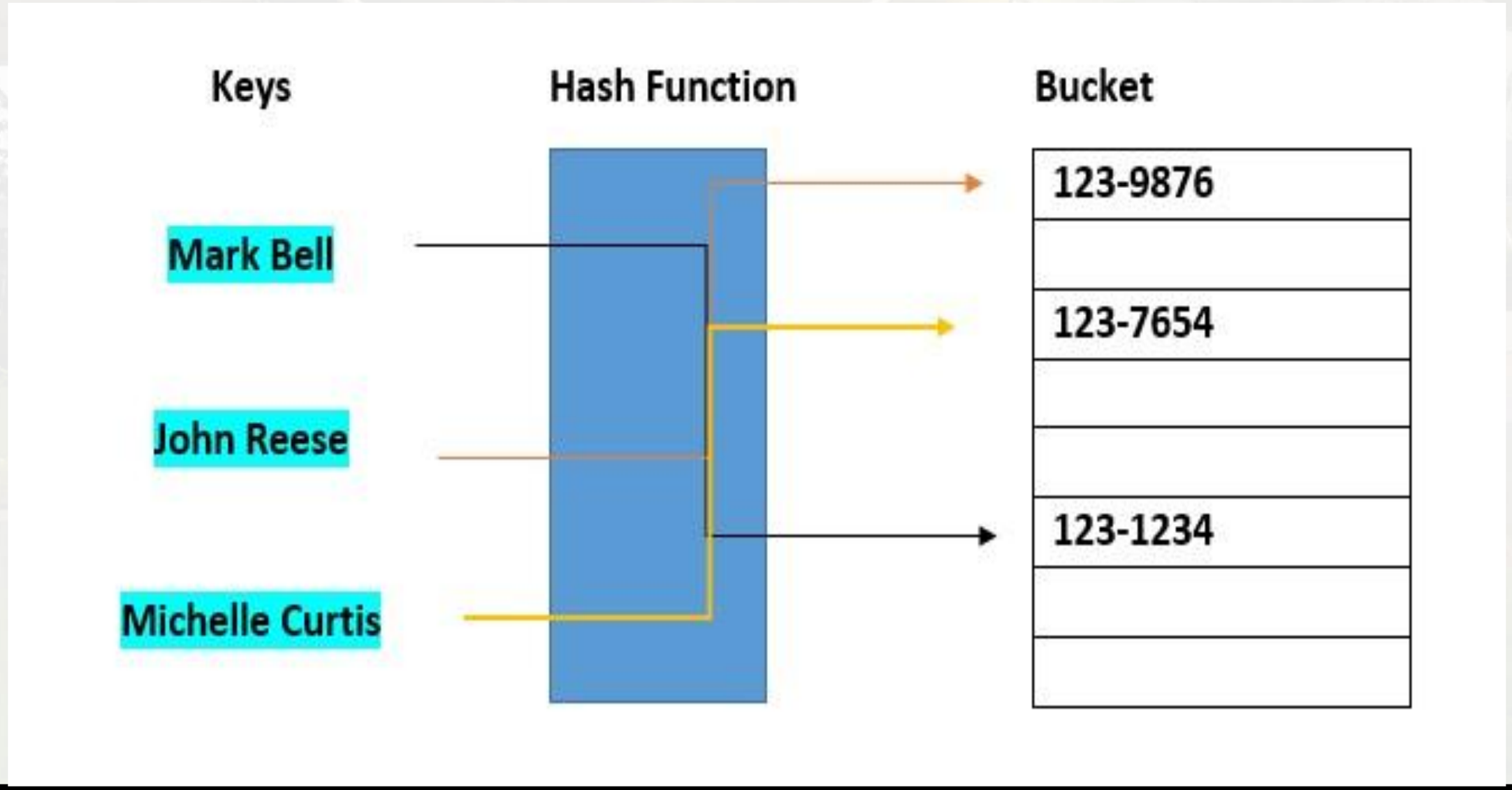
- **A function** that maps a key into the range [0 to Max − 1], the result of which is used as an index (or address) to hash table for storing and retrieving record

- The address generated by hashing function is called as **home address**

- All home addresses address to particular area of memory and that area is called **as prime area**

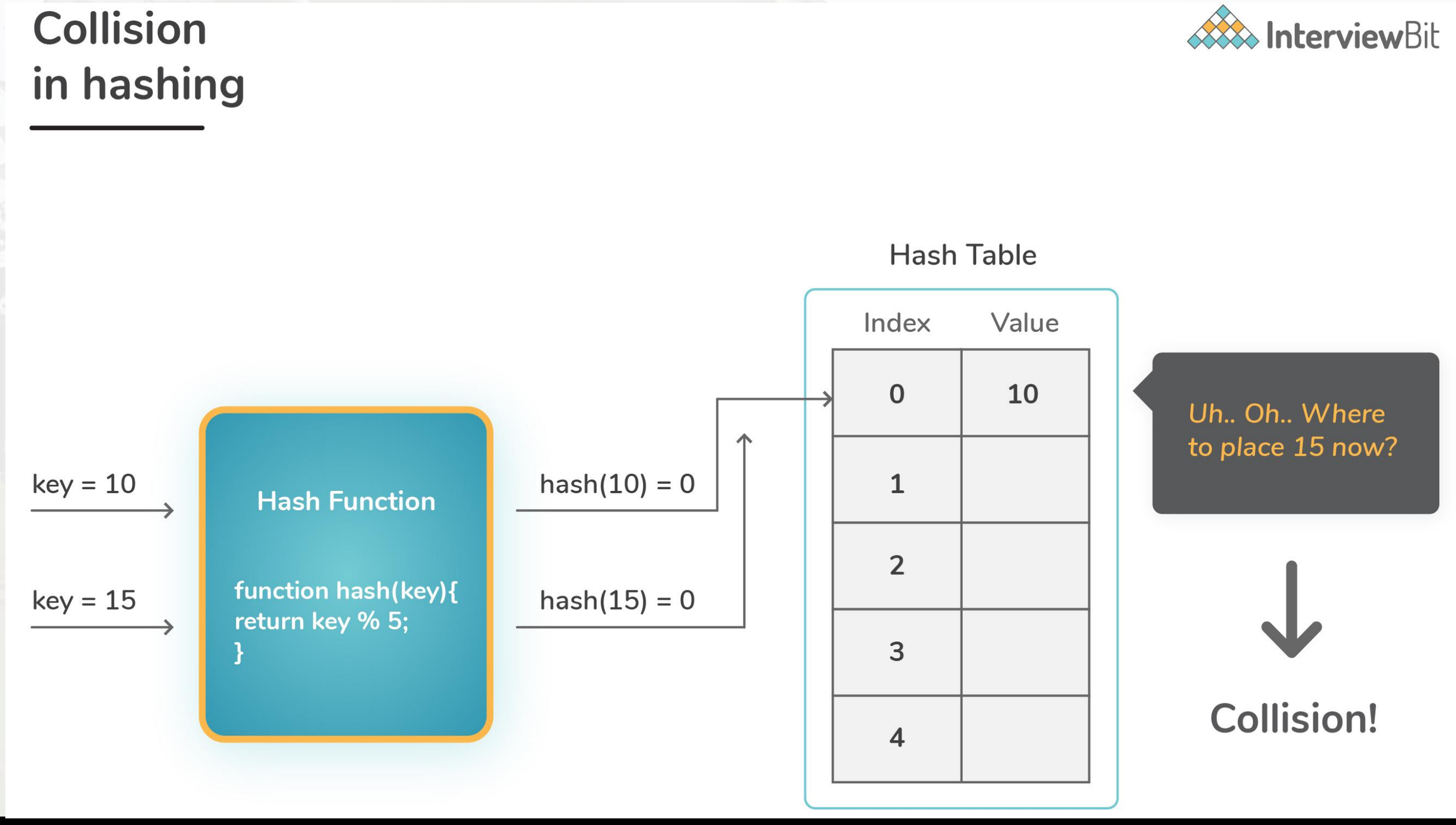
BUCKET

- **Bucket** is an index position in hash table that can store more than one record
- When the same index is mapped with two keys, then both the records are stored in the same bucket



COLLISION

- The result of two keys hashing into the same address is called **collision**



PROBE

- Each calculation of an address and test for success is known as **Probe.**

SYNONYMS

- Keys those hash to the same address are called **synonyms**

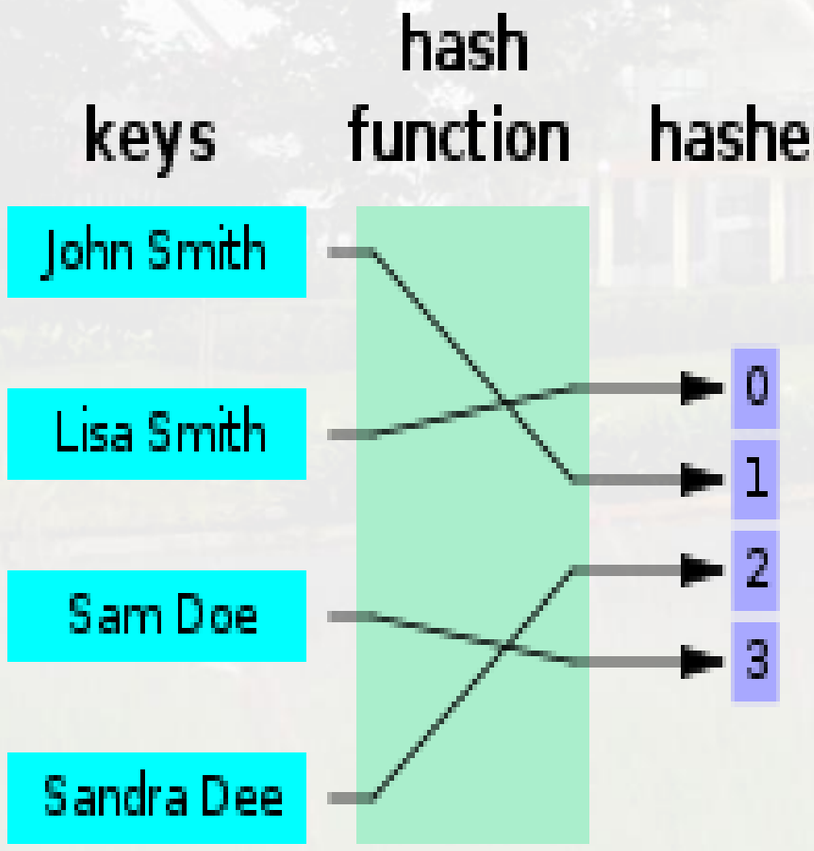
OVERFLOW

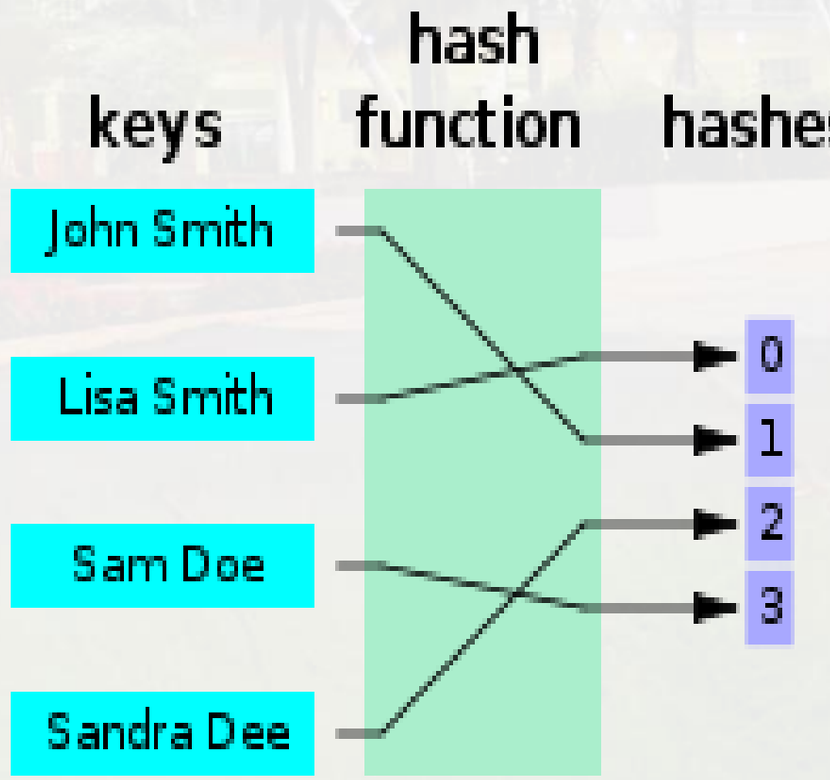
- The result of more keys hashing to the same address and if there is no room in the bucket, then it is said that overflow has occurred

- Collision and overflow are synonymous when the bucket is of size 1

Perfect Hash Function

- A perfect hash function h for a set S is a hash function that maps distinct elements in S to a set of m integers, with no collisions.
- A perfect hash function with values in a limited range can be used for efficient lookup operations, by placing keys from S (or other associated values) in a table indexed by the output of the function.





Perfect Hash Function

- **Advantages :**

1. A perfect hash function with values in a limited range can be used for efficient lookup operations.
2. No need to apply collision resolution techniques.

LOAD FACTOR

Load factor is defined as (m/n) where n is the total size of the hash table and m is the preferred number of entries which can be inserted before a increment in size of the underlying data structure is required.

$$\text{Load Factor } (\alpha) = \frac{\text{Number of elements present in the hash table}}{\text{Total size of the hash table}}$$

If Load factor (α) = constant, then time complexity of Insert, Search, Delete = Θ(1)

Load factor and Load density

- **Load Factor:** The ratio of the number of items in a table to the table's size is called the load factor.

- **Load Density :** The identifier density of a hash table is the ratio n/T,
- where **n is the number of identifiers in the table**. The loading density or loading factor of a hash table is a=n /(sb).
- T is total number of possible element.

Load factor and Load density

- **Load Density :** The identifier density of a hash table is the ratio n/T,
- **Example :**
- Consider the hash table with b = 26 buckets and s = 2.

  We have n = 10 distinct identifiers, each representing a C library function.
- This table has a loading factor, a, of **10/52 = 0.19**

HASHING

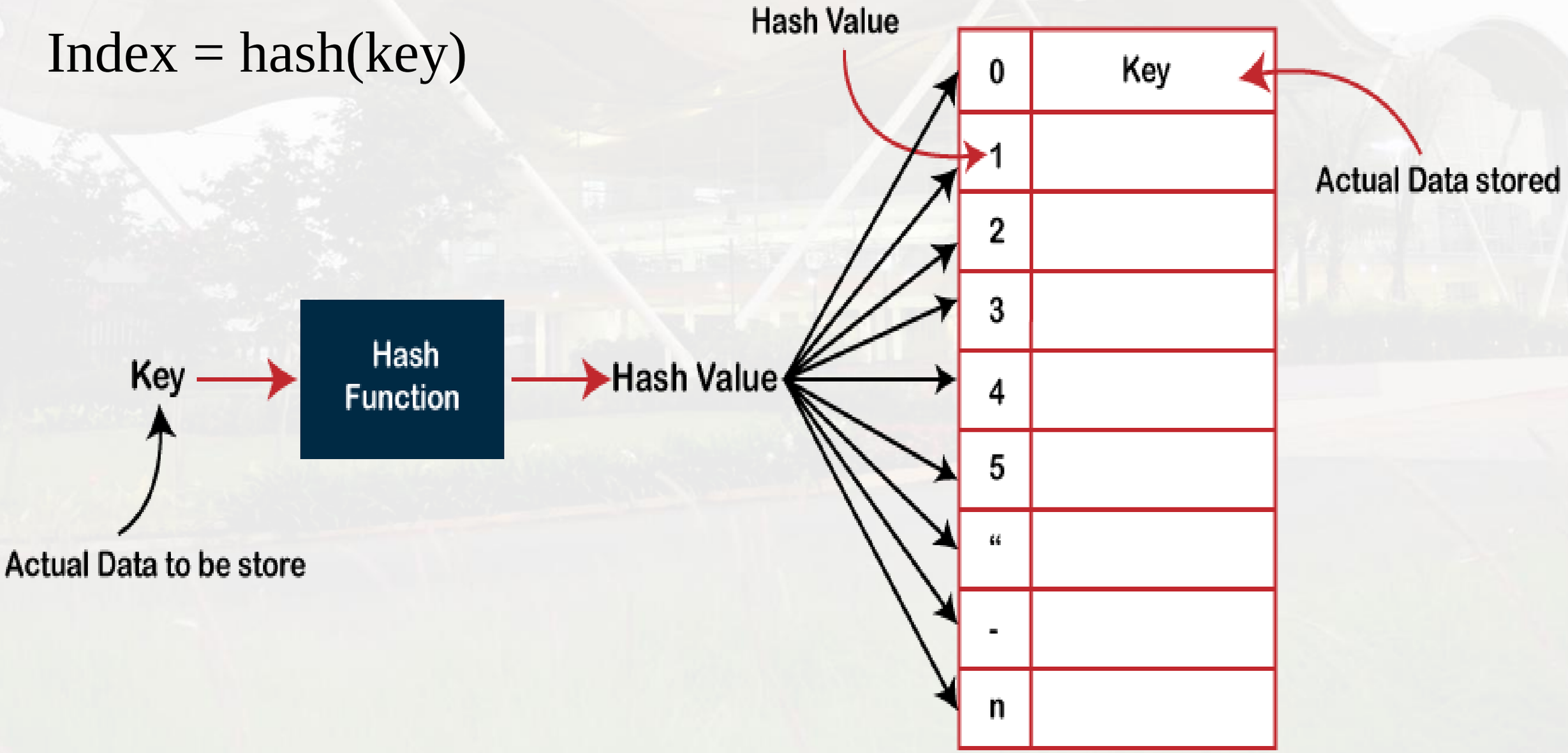
- **Hashing** is one of the searching techniques that uses a constant time. The time complexity in hashing is O(1). Till now, we read the two techniques for searching, i.e., linear search and binary search

- The worst time complexity in linear search is O(n), and O(logn) in binary search. In both the searching techniques, the searching depends upon the number of elements but we want the technique that takes a constant time. So, hashing technique came that provides a constant time.

- **In Hashing technique**, the hash table and hash function are used. Using the hash function, we can calculate the address at which the value can be stored.

HASHING/HASH FUNCTION

- The main idea behind the hashing is to create the (key/value) pairs. If the key is given, then the algorithm computes the index at which the value would be stored. It can be written as:
- Index = hash(key)



TYPES OF HASH FUNCTION

a) Division Method

b) Multiplication Method

c) Extraction Method

d) Mid square method

e) Folding

f) Universal Method

TYPES OF HASH FUNCTION

There are three ways of calculating the hash function:

1. Division method
2. Folding method
3. Mid square method

In the division method, the hash function can be defined as:

h(ki) = ki % m;

where m is the size of the hash table.

For example, if the key value is 6 and the size of the hash table is 10. When we apply the hash function to key 6 then the index would be:

h(6) = 6%10 = 6

The index is 6 at which the value is stored.

TYPES OF HASH FUNCTION

1. Division Method:

This is the most simple and easiest method to generate a hash value. The hash function divides the value k by M and then uses the remainder obtained.

Formula:

h(K) = k mod M

Here,

k is the key value, and

M is the size of the hash table.

Example:

k = 12345

M = 95

h(12345) = 12345 mod 95

= 90

k = 1276

M = 11

h(1276) = 1276 mod 11

= 0

TYPES OF HASH FUNCTION

2. The mid square method is a very good hashing method. It involves two steps to compute the hash value-

Square the value of the key k i.e. k^2

Extract the middle r digits as the hash value**.**

Formula:

h(K) = h(k x k)

Here,

k is the key value.

Example:

Suppose the hash table has 100 memory locations. So r = 2 because two digits are required to map the key to the memory location.

k = 60

k x k = 60 x 60

= 3**60**0 (mid 60 for k=60)

h(60) = 60

The hash value obtained is 60

TYPES OF HASH FUNCTION

3. Digit Folding Method : This method involves two steps:

Divide the key-value k into a number of parts i.e. k1, k2, k3,….,kn, where each part has the same number of digits except for the last part that can have lesser digits than the other parts.

Add the individual parts. The hash value is obtained by ignoring the last carry if any.

Formula:

k = k1, k2, k3, k4, ….., kn

s = k1+ k2 + k3 + k4 +….+ kn

h(K)= s

Here,

s is obtained by adding the parts of the key k

TYPES OF HASH FUNCTION

3. Digit Folding Method :

Example:

k = 12345

k1 = 12, k2 = 34, k3 = 5

s = k1 + k2 + k3

= 12 + 34 + 5

= 51

h(K) = 51

TYPES OF HASH FUNCTION

4. Multiplication method :

This method involves the following steps:

- Choose a constant value A such that $0 < A < 1$.
- Multiply the key value with A.
- Extract the fractional part of kA.
- Multiply the result of the above step by the size of the hash table i.e. M.
- The resulting hash value is obtained by taking the floor of the result obtained in step 4.

TYPES OF HASH FUNCTION

4. Multiplication method :

Formula:

$$h(K) = \text{floor}\,(M\,(kA \bmod 1))$$

Here,

M is the size of the hash table.

k is the key value.

A is a constant value.

Where "k A mod 1" means the **fractional part of k A,** that is, k A **-**⌊k A⌋.

TYPES OF HASH FUNCTION

4. Multiplication method :

Example:

k = 12345

A = 0.357840

M = 100

h(12345) = floor[100 (12345*0.357840 mod 1)]

= floor[100 (4417.5348 mod 1)]

= floor[100 (0.5348)]

= floor[53.48]

= 53

Collision in Hashing

- **Collision in Hashing-**

In hashing,

1. Hash function is used to compute the hash value for a key.
2. Hash value is then used as an index to store the key in the hash table.
3. Hash function may return the same hash value for two or more keys.

“When the hash value of a key maps to an already occupied bucket of the hash table, it is called as a Collision”

COLLISION

When the two different values have the same value, then the problem occurs between the two values, known as a collision. In the above example, the value is stored at index 6. If the key value is 26, then the index would be:

h(26) = 26%10 = 6

Therefore, two values are stored at the same index, i.e., 6, and this leads to the collision problem. To resolve these collisions, we have some techniques known as collision techniques.

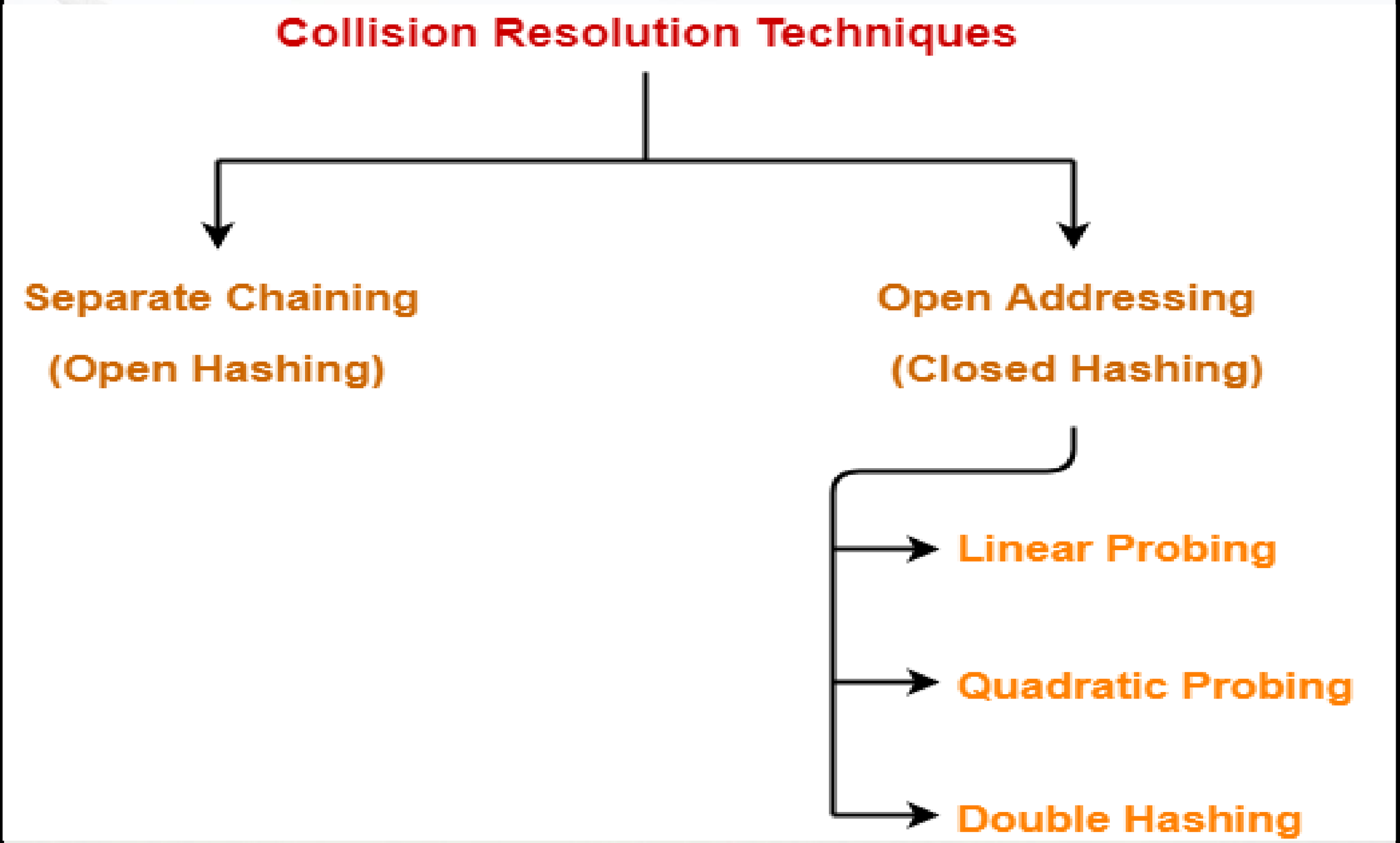
The following are the collision techniques:

Open Hashing: It is also known as closed addressing.

Closed Hashing: It is also known as open addressing.

Collision Resolution Techniques

In Hashing, collision resolution techniques are classified as-



Separate Chaining

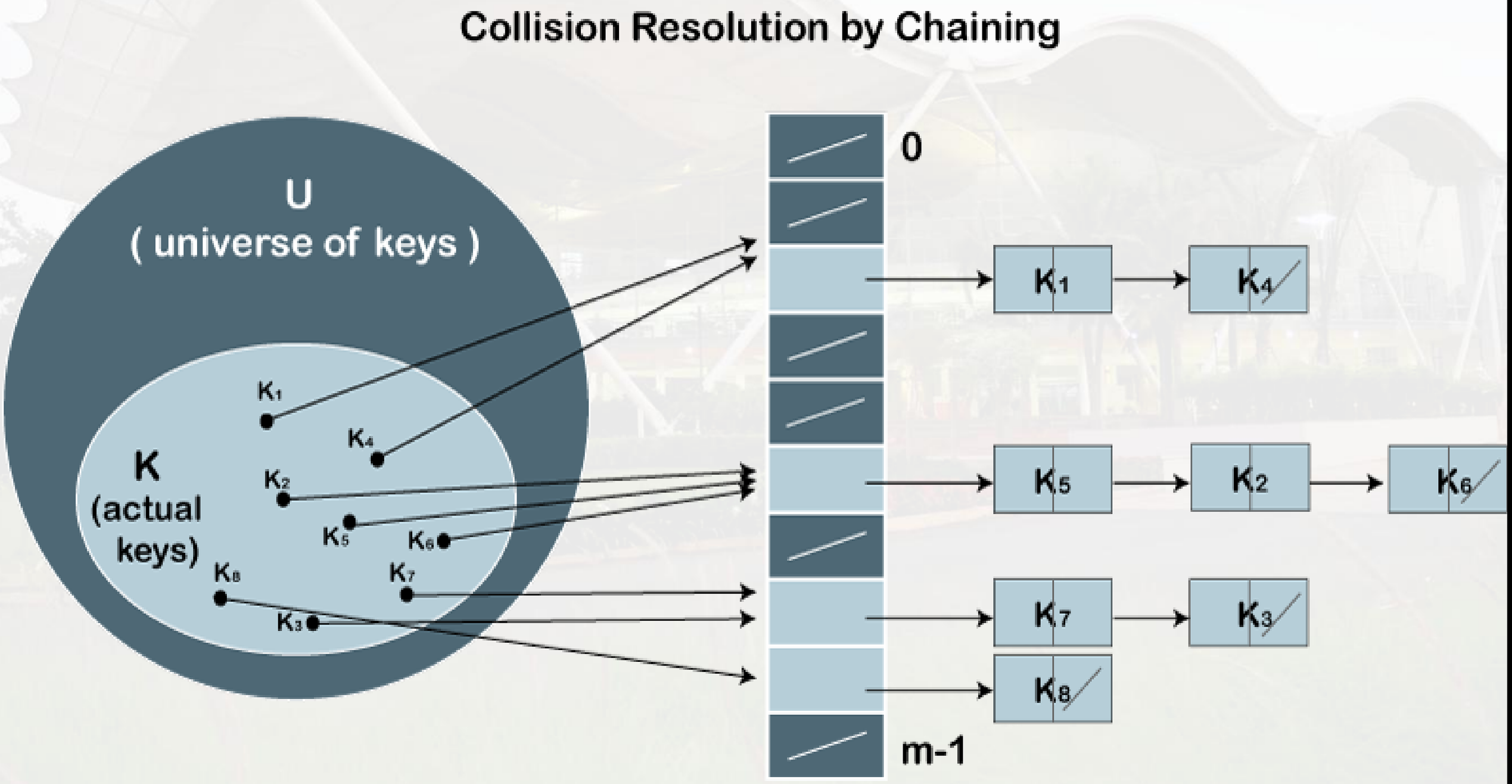
(Open hashing/External hashing)

OPEN HASHING

The first Collision Resolution or Handling technique, " Open Hashing ", is popularly known as Separate Chaining. This is a technique which is used to implement an array as a linked list known as a chain. It is one of the most used techniques by programmers to handle collisions. Basically, a linked list data structure is used to implement the Separate Chaining technique. When a number of elements are hashed into the index of a single slot, then they are inserted into a singly-linked list. This singly-linked list is the linked list which we refer to as a chain in the Open Hashing technique.

OPEN HASHING

n Open Hashing, one of the methods used to resolve the collision is known as a chaining method.



Collision Resolution Techniques

1. Separate Chaining (hashing with chaining/open hashing)

To handle the collision,

1. This technique creates a linked list to the slot for which collision occurs.
2. The new key is then inserted in the linked list.
3. These linked lists to the slots appear like chains.
4. That is why, this technique is called as separate chaining

Collision Resolution Techniques

For Searching-

- In worst case, all the keys might map to the same bucket of the hash table.
- In such a case, all the keys will be present in a single linked list.
- Sequential search will have to be performed on the linked list to perform the search.
- So, time taken for searching in worst case is O(n).

For Deletion-

- In worst case, the key might have to be searched first and then deleted.
- In worst case, time taken for searching is O(n).
- So, time taken for deletion in worst case is O(n).

Collision Resolution Techniques

Advantages:

1) Simple to implement.

2) Hash table never fills up, we can always add more elements to the chain.

3) Less sensitive to the hash function or load factors.

4) It is mostly used when it is unknown how many and how frequently keys may be inserted or deleted. .

Collision Resolution Techniques

Disadvantages:

1) Cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.

2) Wastage of Space (Some Parts of hash table are never used)

3) If the chain becomes long, then search time can become O(n) in the worst case.

4) Uses extra space for links

Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

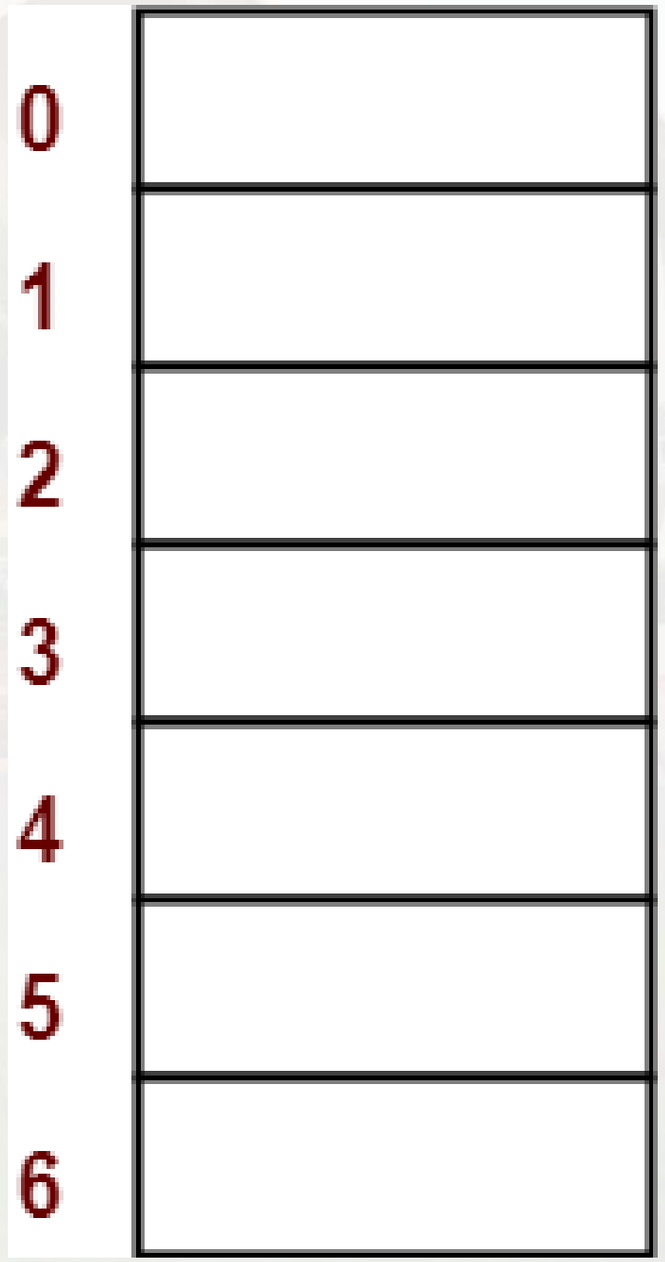
Use separate chaining technique for collision resolution.

Step-01:

Draw an empty hash table.

For the given hash function, the possible range of hash values is [0, 6].

So, draw an empty hash table consisting of 7 buckets as-



Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

Use separate chaining technique for collision resolution.

Step-02:

Insert the given keys in the hash table one by one.

The first key to be inserted in the hash table = 50.

Bucket of the hash table to which key 50 maps = 50 mod 7 = 1.

So, key 50 will be inserted in bucket-1 of the hash table as-

0	
1	50
2	
3	
4	
5	
6	

Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

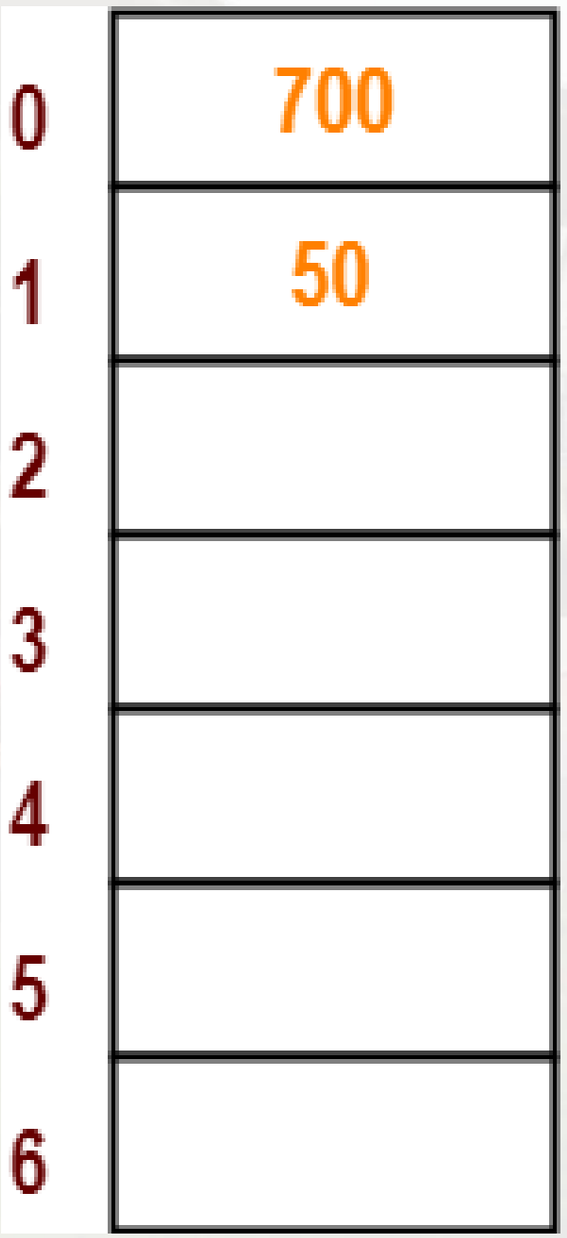
Use separate chaining technique for collision resolution.

Step-03:

The next key to be inserted in the hash table = 700.

Bucket of the hash table to which key 700 maps = 700 mod 7 = 0.

So, key 700 will be inserted in bucket-0 of the hash table as-



Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

Use separate chaining technique for collision resolution.

Step-04:

The next key to be inserted in the hash table = 76.

Bucket of the hash table to which key 76 maps = 76 mod 7 = 6.

So, key 76 will be inserted in bucket-6 of the hash table as-

0	700
1	50
2	
3	
4	
5	
6	76

Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

Use separate chaining technique for collision resolution.

Step-05:

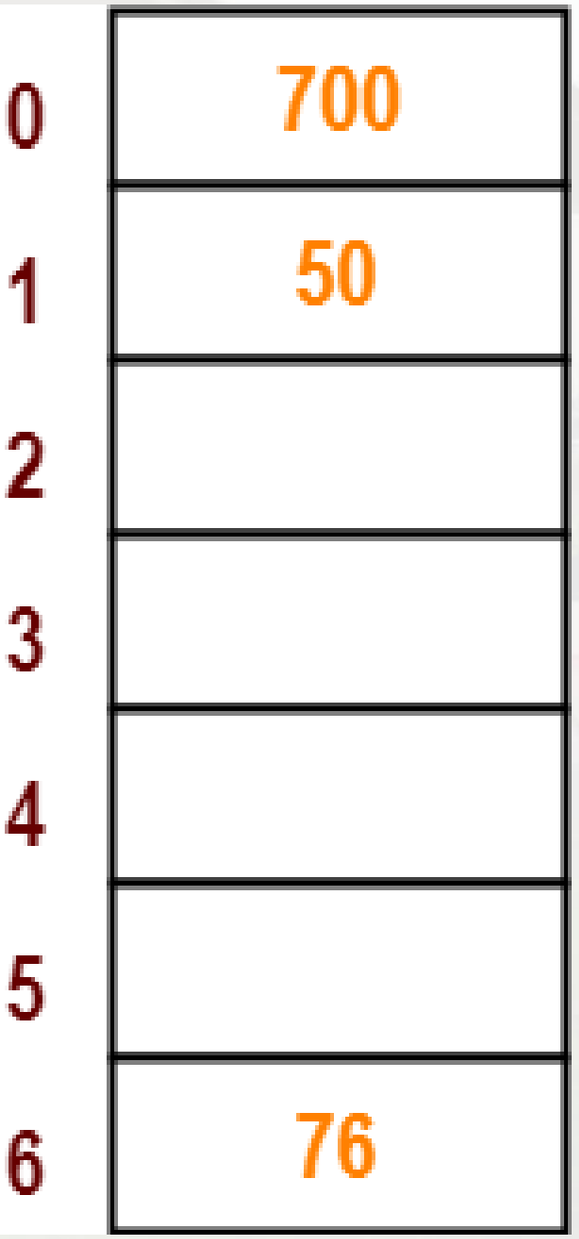
- The next key to be inserted in the hash table = 85.

Bucket of the hash table to which key 85 maps = 85 mod 7 = 1.

Since bucket-1 is already occupied, so collision occurs.

Separate chaining handles the collision by creating a linked list to bucket-1.

So, key 85 will be inserted in bucket-1 of the hash table as-



Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

Use separate chaining technique for collision resolution.

Step-06 :

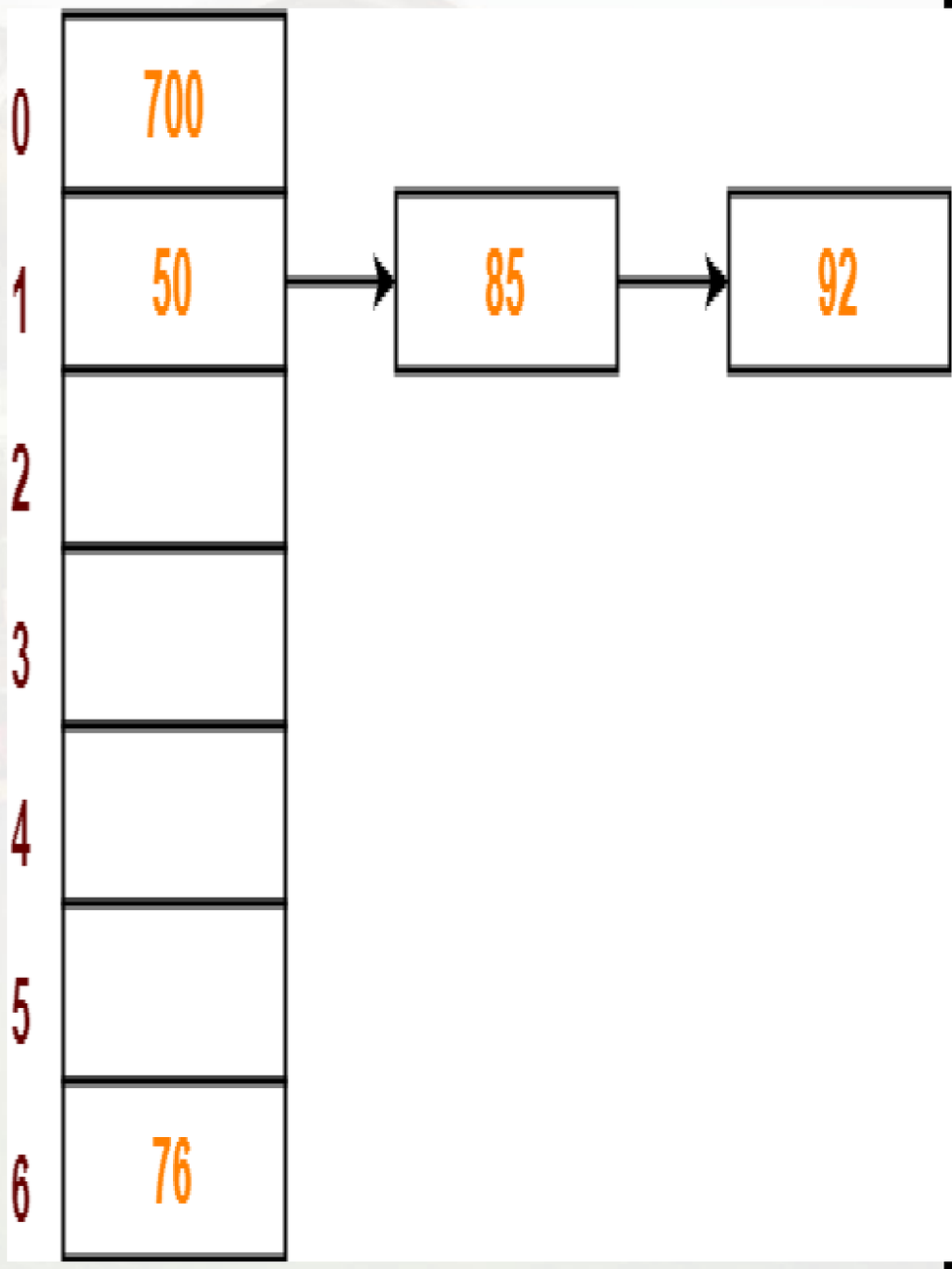
The next key to be inserted in the hash table = 92.

Bucket of the hash table to which key 92 maps = 92 mod 7 = 1.

Since bucket-1 is already occupied, so collision occurs.

Separate chaining handles the collision by creating a linked list to bucket-1.

So, key 92 will be inserted in bucket-1 of the hash table as-



Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

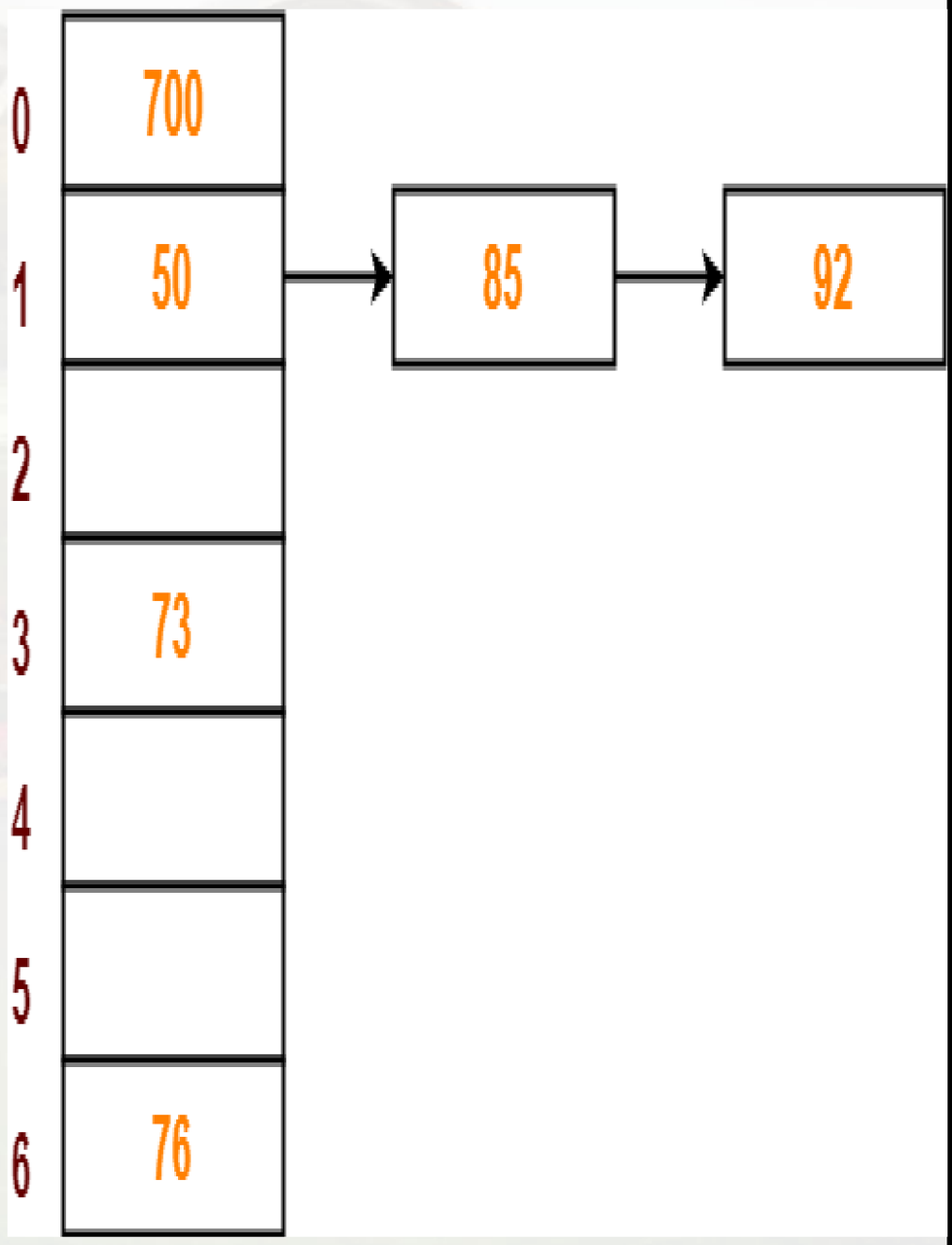
Use separate chaining technique for collision resolution.

Step-07 :

The next key to be inserted in the hash table = 73.

Bucket of the hash table to which key 73 maps = 73 mod 7 = 3.

So, key 73 will be inserted in bucket-3 of the hash table as-



Collision Resolution Techniques

Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

Use separate chaining technique for collision resolution.

Step-08 :

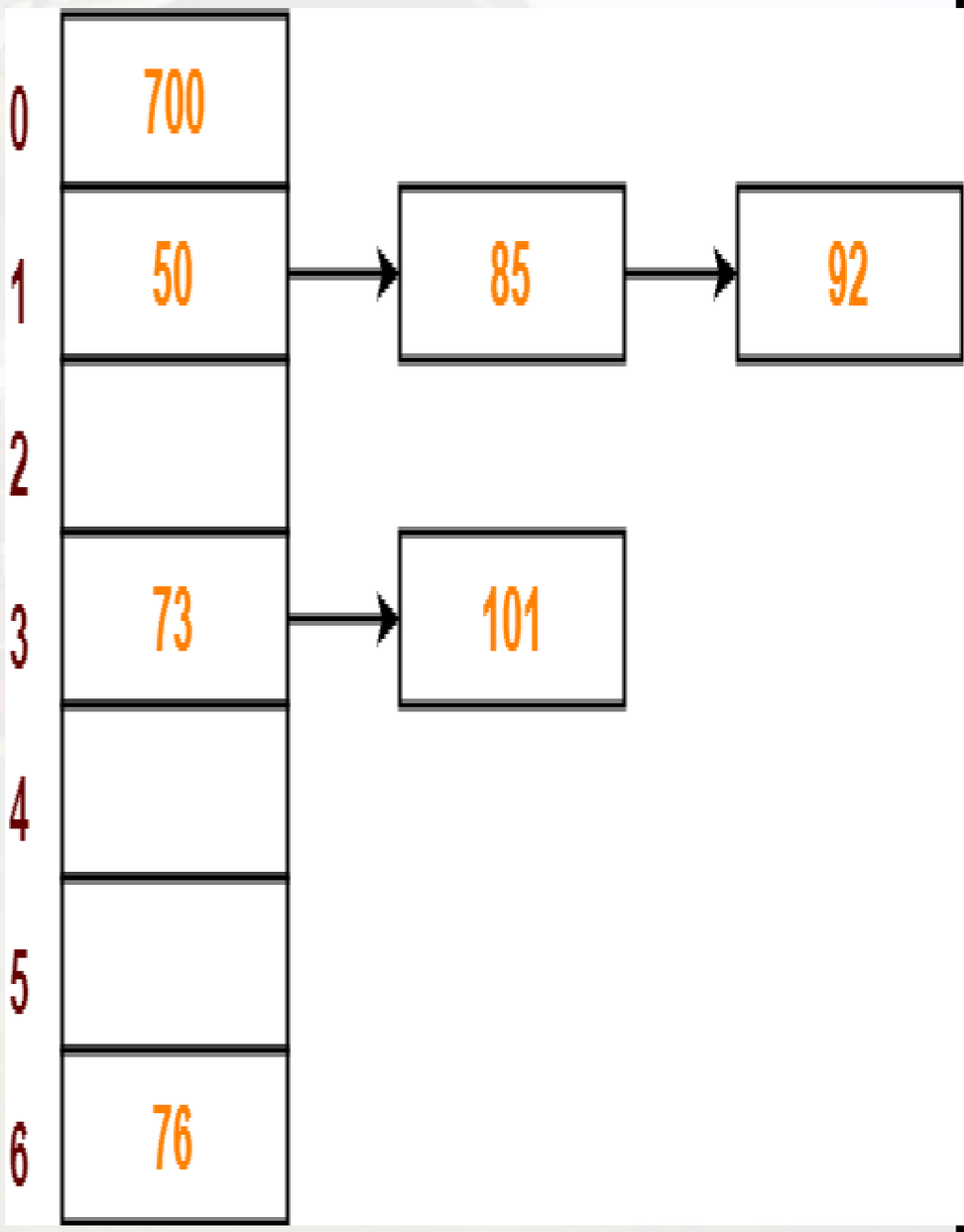
The next key to be inserted in the hash table = 101.

Bucket of the hash table to which key 101 maps = 101 mod 7 = 3.

Since bucket-3 is already occupied, so collision occurs.

Separate chaining handles the collision by creating a linked list to bucket-3.

So, key 101 will be inserted in bucket-3 of the hash table as-

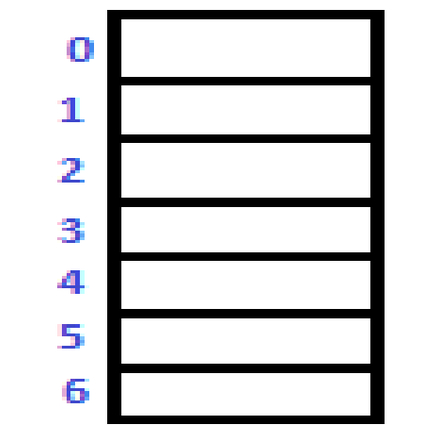


Collision Resolution Techniques

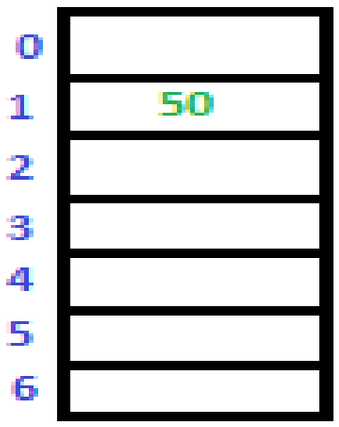
Using the hash function 'key mod 7', insert the following sequence of keys in the hash table-

50, 700, 76, 85, 92, 73 and 101.

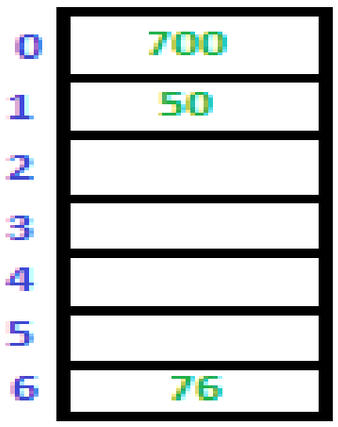
Use separate chaining technique for collision resolution.



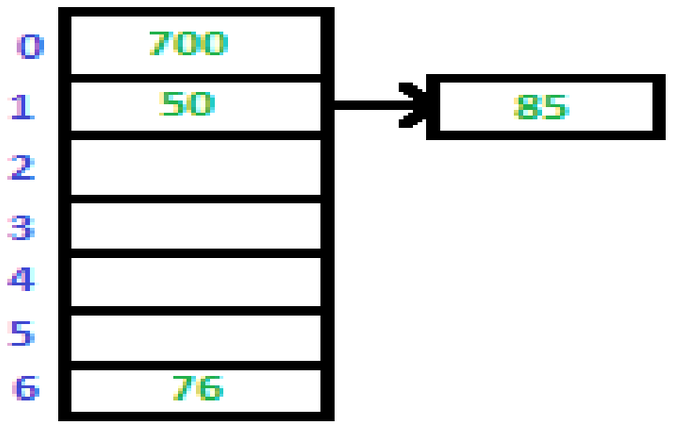
Initial Empty Table



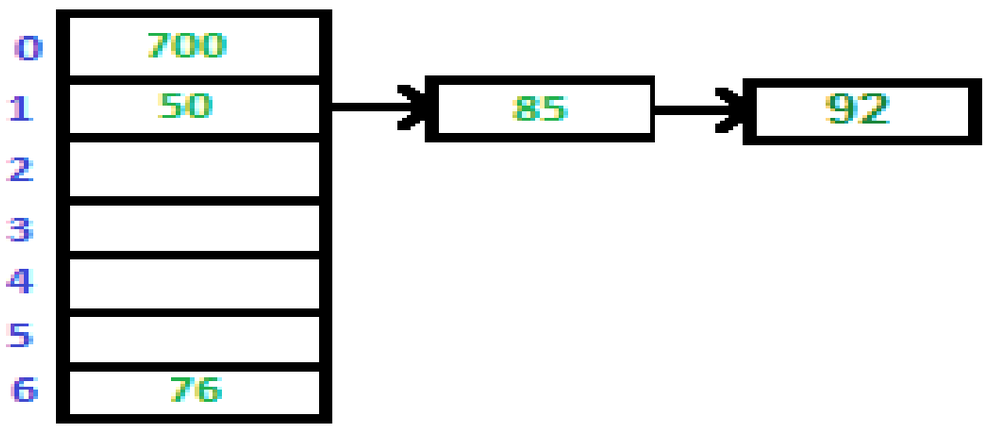
Insert 50



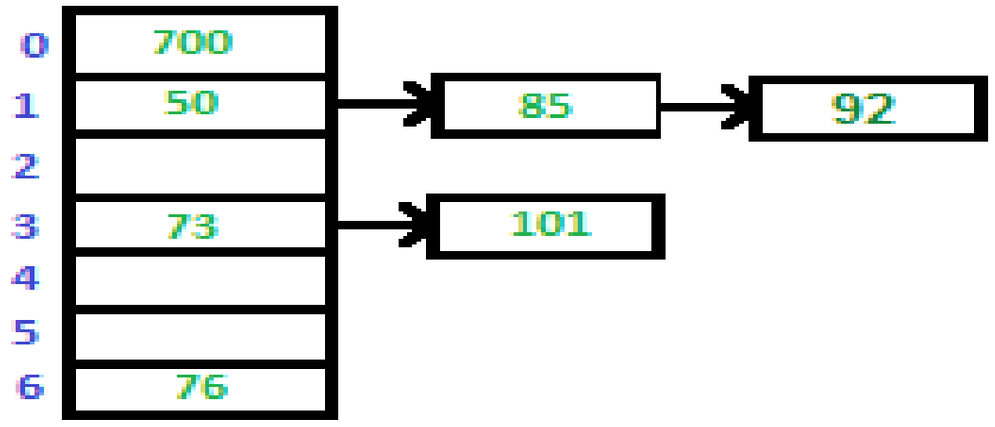
Insert 700 and 76



Insert 85: Collision
Occurs, add to chain



Inser 92 Collision
Occurs, add to chain



Insert 73 and 101

OPEN HASHING

Let's first understand the chaining to resolve the collision.

Suppose we have a list of key values

A = 3, 2, 9, 6, 11, 13, 7, 12 where m = 10, and h(k) = 2k+3

In this case, we cannot directly use h(k) = ki/m as h(k) = 2k+3

The index of key value 3 is:

index = h(3) = (2(3)+3)%10 = 9

The value 3 would be stored at the index 9.

The index of key value 2 is:

index = h(2) = (2(2)+3)%10 = 7

The value 2 would be stored at the index 7.

OPEN HASHING

The index of key value 9 is:

index = h(9) = (2(9)+3)%10 = 1

The value 9 would be stored at the index 1.

The index of key value 6 is:

index = h(6) = (2(6)+3)%10 = 5

The value 6 would be stored at the index 5.

The index of key value 11 is:

index = h(11) = (2(11)+3)%10 = 5

The value 11 would be stored at the index 5.

OPEN HASHING

The index of key value 13 is:

index = h(13) = (2(13)+3)%10 = 9

The value 13 would be stored at index 9.

The index of key value 7 is:

index = h(7) = (2(7)+3)%10 = 7

The value 7 would be stored at index 7.

The index of key value 12 is:

index = h(12) = (2(12)+3)%10 = 7

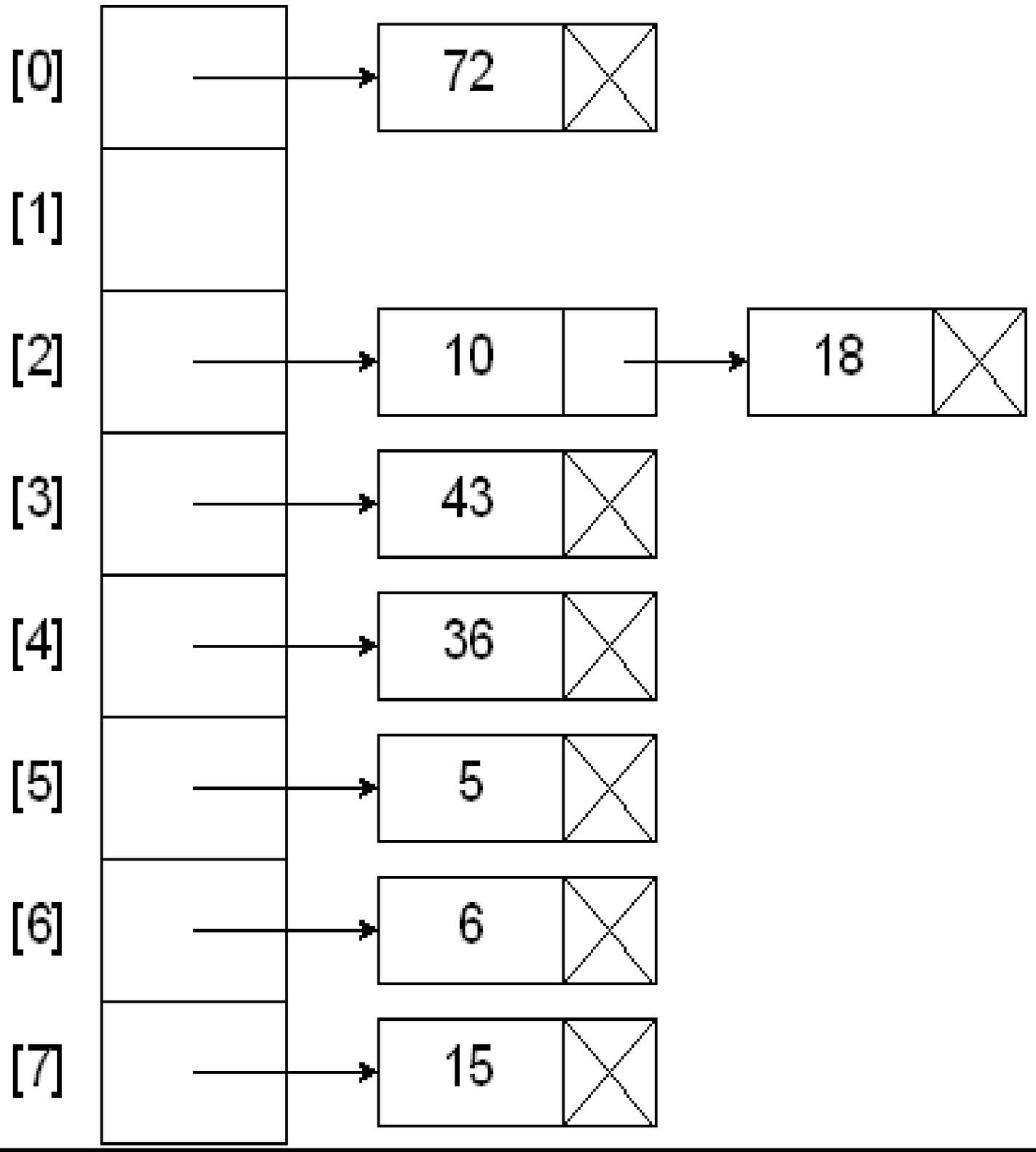
The value 7 would be stored at index 7.

OPEN HASHING

The hash table slots will no longer hold a table element. They will now hold the address of a table element.

Hash key = key % table size

4 = 36 % 8
2 = 18 % 8
0 = 72 % 8
3 = 43 % 8
6 = 6 % 8
2 = 10 % 8
5 = 5 % 8
7 = 15 % 8



OPEN HASHING

$$36 \% 8 = 4$$

$$18 \% 8 = 2$$

$$72 \% 8 = 0$$

$$43 \% 8 = 3$$

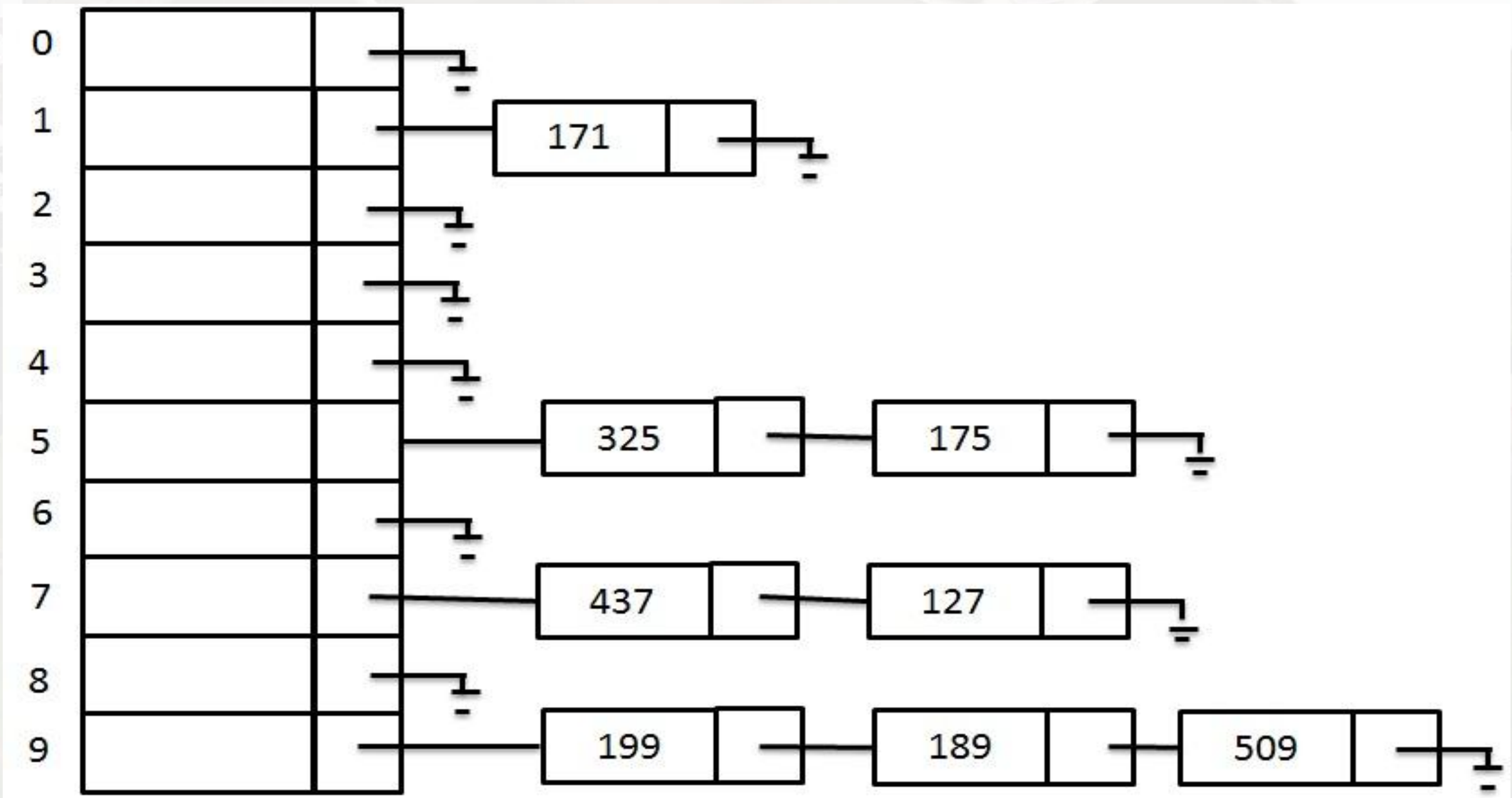
$$6 \% 8 = 6$$

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
72		18	43	36		6	

OPEN HASHING

Let us say that we have a sequence of numbers { 437, 325, 175, 199, 171, 189, 127, 509} and a hash function H(X) = X mod 10

Let us see the results of separate chaining hash table.



OPEN HASHING

```
For first key: 437
                H (437) = 437 % 10 = 7
                The index for 437 is 7.
For key: 325
                  H (325) = 325 % 10 = 5
                  The index for 325 is 5.
For key: 175
                  H (175) = 175 % 10 = 5
                  The index for 175 is 5.
For key: 199
                H (199) = 199 % 10 = 9
                The index for 199 is 9.
For key: 171
               H (171) = 171 % 10 = 1
               The index for 171 is 1.
For key: 189
                 H (189) = 189 % 10 = 9
                 The index for 189 is 9.
For key: 127
                H (127) = 127 % 10 = 7
                The index for 127 is =7.
For key: 509
                H (509) = 509 % 10 = 9
                The index for 509 is 9.
```

Open addressing

(Close hashing/Internal hashing)

CLOSED HASHING

In Closed hashing, three techniques are used to resolve the collision:

1. Linear probing
2. Quadratic probing
3. Double Hashing technique

PROBING

Method	Description
Linear probing	Just like the name suggests, this method searches for empty slots linearly starting from the position where the collision occurred and moving forward. If the end of the list is reached and no empty slot is found. The probing starts at the beginning of the list.
Quadratic probing	This method uses quadratic polynomial expressions to find the next available free slot.
Double Hashing	This technique uses a secondary hash function algorithm to find the next free available slot.

Open Addressing

In open addressing,

1. Unlike separate chaining, all the keys are stored inside the hash table.
2. No key is stored outside the hash table.

Techniques used for open addressing are-

1. Linear Probing
2. Quadratic Probing
3. Double Hashing

Open Addressing

Operations In open addressing,

1. Insert Operation-

- Hash function is used to compute the hash value for a key to be inserted.
- Hash value is then used as an index to store the key in the hash table.

In case of collision,

- Probing is performed until an empty bucket is found.
- Once an empty bucket is found, the key is inserted.
- Probing is performed in accordance with the technique used for open addressing.

Open Addressing

Operations In open addressing,

Search Operation-

To search any particular key,

- Its hash value is obtained using the hash function used.
- Using the hash value, that bucket of the hash table is checked.
- If the required key is found, the key is searched.
- Otherwise, the subsequent buckets are checked until the required key or an empty bucket is found.
- The empty bucket indicates that the key is not present in the hash table.

Open Addressing

Operations In open addressing,

Search Operation-

- The key is first searched and then deleted.
- After deleting the key, that particular bucket is marked as "deleted".

Open Addressing Techniques

1. Linear Probing-

1. When collision occurs, we linearly probe for the next bucket.
2. We keep probing until an empty bucket is found.

Advantage-

1. It is easy to compute.

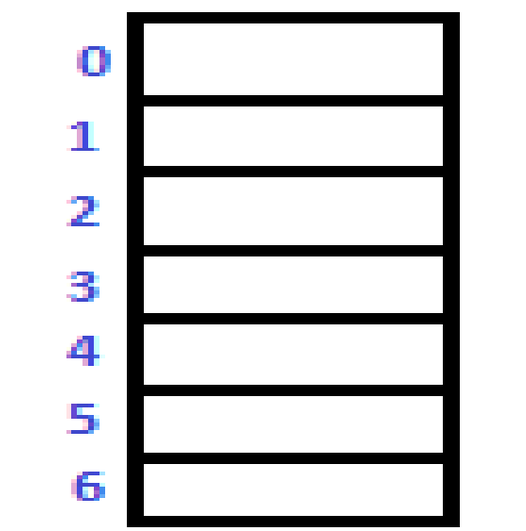
Disadvantage-

1. The main problem with linear probing is clustering.
2. Many consecutive elements form groups.
3. Then, it takes time to search an element or to find an empty bucket.

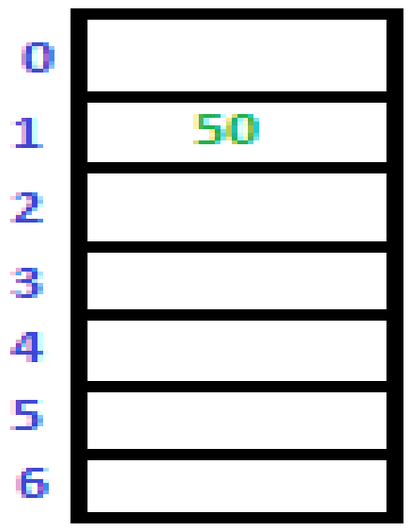
Open Addressing Techniques

1. Linear Probing -

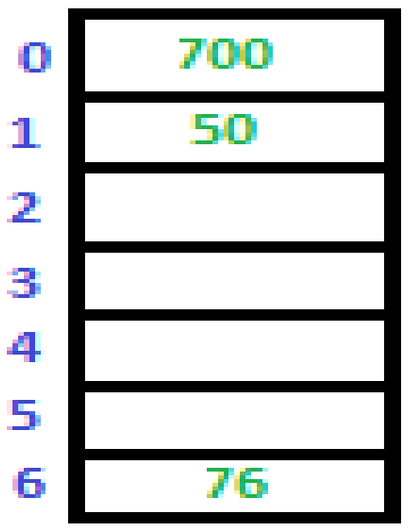
Let us consider a simple hash function as “key mod 7” and a sequence of keys as 50, 700, 76, 85, 92, 73, 101.



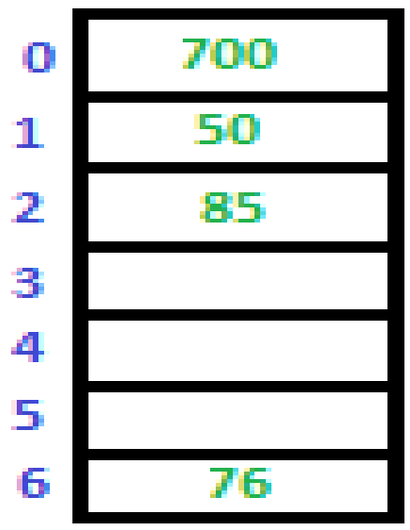
Initial Empty Table



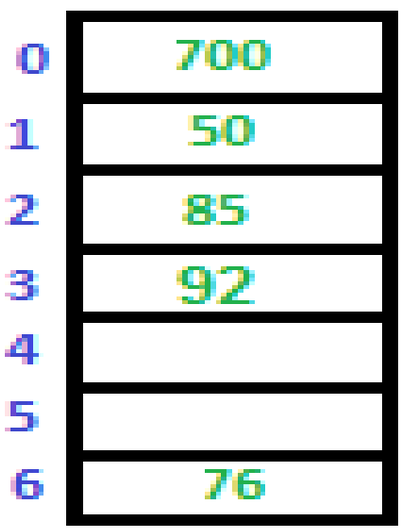
Insert 50



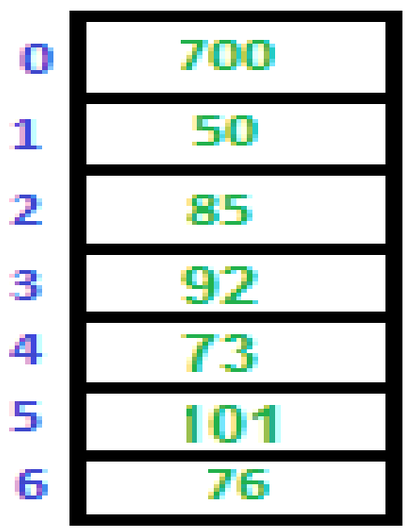
Insert 700 and 76



Insert 85: Collision Occurs, insert 85 at next free slot.



Insert 92, collision occurs as 50 is there at index 1. Insert at next free slot



Insert 73 and 101

CLOSED HASHING

Linear Probing

Linear probing is one of the forms of open addressing. As we know that each cell in the hash table contains a key-value pair, **so when the collision occurs by mapping a new key to the cell already occupied by another key, then linear probing technique searches for the closest free locations and adds a new key to that empty cell**. In this case, searching is performed sequentially, starting from the position where the collision occurs till the empty cell is not found.

CLOSED HASHING

Let's understand the linear probing through an example.

Consider the above example for the linear probing:

A = 3, 2, 9, 6, 11, 13, 7, 12 where m = 10, and h(k) = 2k+3

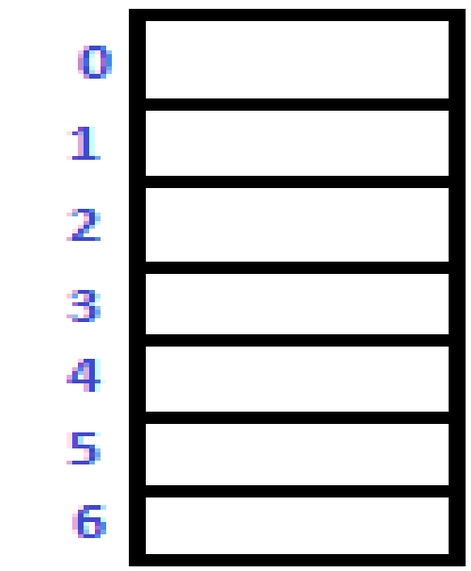
The key values 3, 2, 9, 6 are stored at the indexes 9, 7, 1, 5 respectively.

The calculated index value of 11 is 5 which is already occupied by another key value, i.e., 6. When linear probing is applied, the nearest empty cell to the index 5 is 6; therefore, the value 11 will be added at the index 6.

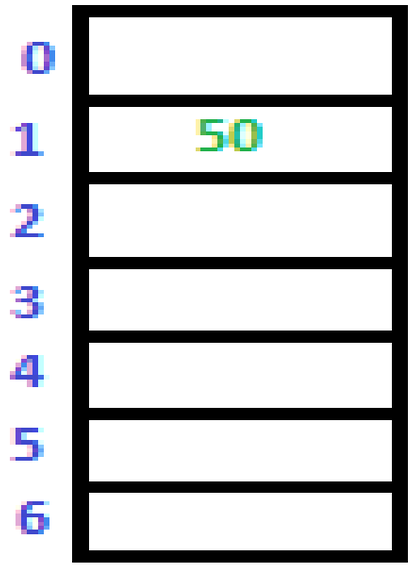
The next key value is 13. The index value associated with this key value is 9 when hash function is applied. The cell is already filled at index 9. When linear probing is applied, the nearest empty cell to the index 9 is 0; therefore, the value 13 will be added at the index 0

LINEAR PROBING

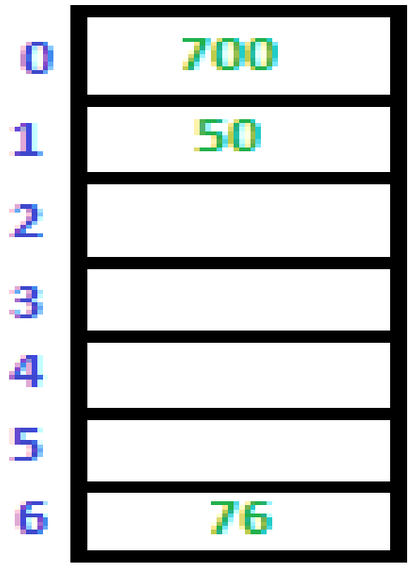
Let us consider a simple hash function as "key mod 7" and a sequence of keys as 50, 700, 76, 85, 92, 73, 101.



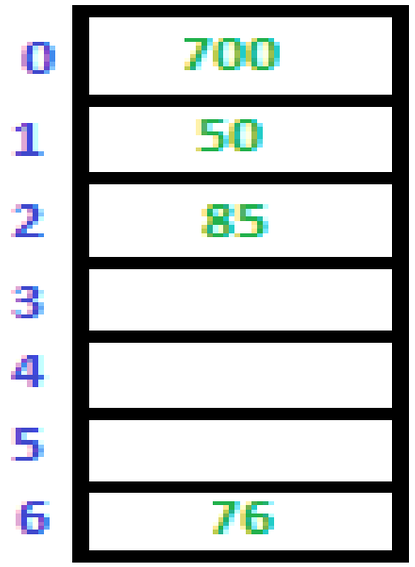
Initial Empty Table



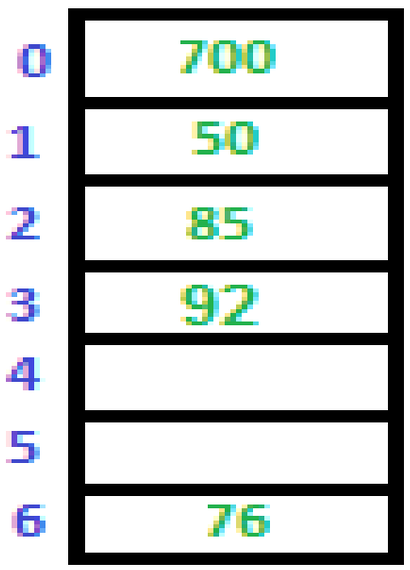
Insert 50



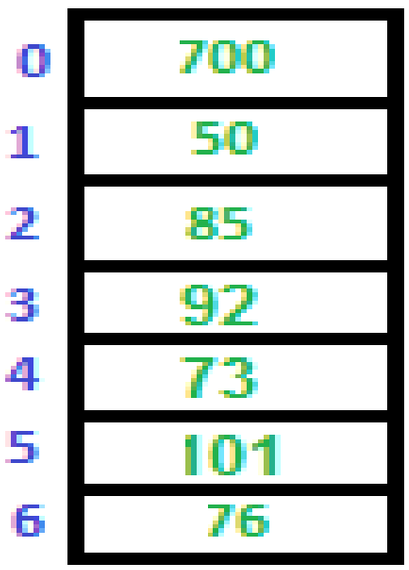
Insert 700 and 76



Insert 85: Collision Occurs, insert 85 at next free slot.



Insert 92, collision occurs as 50 is there at index 1. Insert at next free slot



Insert 73 and 101

LINEAR PROBING

Let us consider a simple hash function as "key mod 30" and a sequence of keys as 3, 1, 63, 5, 11, 15, 18, 16, 46.

Serial Number	Key	Hash	Array Index	Array Index after Linear Probing
1	3	3%30 = 3	3	3
2	1	1%30 = 1	1	1
3	63	63%30 = 3	3	4
4	5	5%30 = 5	5	5
5	11	11%30 = 11	11	11
6	15	15%30 = 15	15	15
7	18	18%30 = 18	18	18
8	16	16%30 = 16	16	16
9	46	46%30 = 8	16	17

Chaining

(with /without replacement)

Chaining Without Replacement

In collision handling method chaining is a concept which introduces an additional field with data i.e. chain. A separate chain table is maintained for colliding data. When collision occurs, we store the second colliding data by **linear probing method**. The address of this colliding data can be stored with the first colliding element in the chain table, without replacement.

Chaining Without Replacement

For example, consider elements:

131, 3, 4, 21, 61, 6, 71, 8, 9

Index	Data	Chain
0	-1	-1
1	131	2
2	21	5
3	3	-1
4	4	-1
5	61	7
6	6	-1
7	71	-1
8	8	-1
9	9	-1

Chaining With Replacement

For example, consider elements:

Identifier is 11, 32, 41, 54, 33. Hash function: f(x) = X mod 10 and Table size = 10

	X	Chain
0		-1
1	11	3
2	32	-1
3	41	5
4	54	-1
5	33	-1
-	-	
-	-	
-	-	
9		

Chaining With Replacement

① Chaining without Replacement

eg : 131, 3, 4, 21, 61, 6, 71, 8, 9

1.) 131 % 10 = 1

2.) 3 % 10 = 3

3.) 4 % 10 = 4

4.) 21 % 10 = 1 (collision)

5.) 61 % 10 = 1 (collision)

6.) 6 % 10 = 6

7.) 71 % 10 = 1 (collision)

Index	Data	chain
0	-1	-1
1	131	2
2	21	5
3	3	-1
4	4	-1
5	61	7
6	~~71~~ 6	-1
7	71	-1
8	8	-1
9	9	-1

Chaining Without Replacement

For example, consider elements:

Identifier is 11, 32, 41, 54, 33.

Hash function: f(x) = X mod 10 and Table size = 10

	X	Chain
0		-1
1	11	3
2	32	-1
3	41	5
4	54	-1
5	33	-1
.	.	
.	.	
.	.	
9		

Chaining With Replacement

⊙ chaining with Replacement

e.g 131, 21, 31, 4, 5, 2

131 % 10 = 1

21 % 10 = 1

31 % 10 = 1

4 % 10 = 4

5 % 10 = 5

2 % 10 = 2 (collision)

Index	Data	chain
0	-1	-1
1	131	2
2	21	3
3	31	-1
4	4	-1
5	5	-1
6		
7		
8		
9		

Chaining With Replacement

UQ. 1.10.8 Construct hash table of size 10 using linear probing with replacement strategy for collision resolution. The hash function is h (x) = x % 10. Calculate total numbers of comparisons required for searching. Consider slot per bucket is 1.

Bucket	25	3	21	13	1	2	7	12	4	8
0	-1	-1	-1	-1	-1	-1	-1	-1	-1	12 -1
1	-1	-1	21 -1	21-1	21 2	21 6	21 6	21 6	21 6	21 6
2	-1	-1	-1	-1	1 -1	2-1	2 -1	2 8	2 8	2 0
3	-1	3 -1	3 -1	3 4	3 4	3 4	3 4	3 4	3 9	3 9
4	-1	-1	-1	13 -1	13 -1	13 -1	13 -1	13 -1	4 -1	4 -1
5	25 -1	25 -1	25 -1	25 -1	25 -1	25 -1	25 -1	25 -1	25 -1	25 -1
6	-1	-1	-1	-1	-1	1 -1	1 -1	1 -1	1 -1	1 -1
7	-1	-1	-1	-1	-1	-1	7 -1	7 -1	7 -1	7 -1
8	-1	-1	-1	-1	-1	-1	-1	12 -1	12 -1	8 -1
9	-1	-1	-1	-1	-1	-1	-1	-1	13-1	13-1
No. of comparisons	**0**	**0**	**0**	**1**	**1**	**4**	**0**	**6**	**5**	**4**

Chaining With Replacement

UQ. 1.10.9 For the given set of values. 11, 33, 20, 88, 79, 98, 44, 68, 66, 22 Create a hash table with size 10 and resolve collision using chaining with replacement and without replacement. Use the modulus Hash function.

1. **Chaining without replacement**

Data/Chain Bucket	11	33	20	88	79	98	44	68	66	22
0	–1	–1	20 –1	20 –1	20 –1	20 –1	20 –1	20 –1	20 –1	20 –1
1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1
2	–1	–1	–1	–1	–1	98 –1	98 –1	98 5	98 5	98 5
3	–1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1
4	–1	–1	–1	–1	–1	–1	44 –1	44 –1	44 –1	44 –1
5	–1	–1	–1	–1	–1	–1	–1	68 –1	68 –1	68 –1
6	–1	–1	–1	–1	–1	–1	–1	–1	66 –1	66 –1
7	–1	–1	–1	–1	–1	–1	–1	–1	–1	62 –1
8	–1	–1	–1	88 –1	88 –1	88 2	88 2	88 2	88 2	88 2
9	–1	–1	–1	–1	79 –1	79 –1	79 –1	79 –1	79 –1	79 –1

Chaining With Replacement

UQ. 1.10.9 For the given set of values. 11, 33, 20, 88, 79, 98, 44, 68, 66, 22 Create a hash table with size 10 and resolve collision using chaining with replacement and without replacement. Use the modulus Hash function.

2. Chaining with replacement

Data/Chain → Bucket	11	33	20	88	79	98	44	68	66	22
	–1	–1	20 –1	20 –1	20 –1	20 –1	20 –1	20 –1	20 –1	20 –1
0	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1	11 –1
1	–1	–1	–1	–1	–1	98 –1	98 –1	98 5	98 5	22 –1
2	–1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1	33 –1
3	–1	–1	–1	–1	–1	–1	44 –1	44 –1	44 –1	44 –1
4	–1	–1	–1	–1	–1	–1	–1	68 –1	68 –1	68 –1
5	–1	–1	–1	–1	–1	–1	–1	–1	66 –1	66 –1
6	–1	–1	–1	–1	–1	–1	–1	–1	–1	98 –1
7	–1	–1	–1	88 –1	88 –1	88 2	88 2	88 2	88 2	88 5
8	–1	–1	–1	–1	79 –1	79 –1	79 –1	79 –1	79 –1	79 –1

Therefore, we define a new process of Q...

Open Addressing Techniques

2. Quadratic Probing-

- When collision occurs, we probe for i2‘th bucket in ith iteration.
- We keep probing until an empty bucket is found.

CLOSED HASHING

Quadratic Probing

- In case of linear probing, searching is performed linearly. In contrast, quadratic probing is an open addressing technique that uses quadratic polynomial for searching until a empty slot is found.
- It can also be defined as that it allows the insertion ki at first free location from (u+i2)%m where i=0 to m-1.

$\mathbf{h'} = (x) = x \bmod m$

$h(x, i) = (h'(x) + i2) \bmod m$

We can put some other quadratic equations also using some constants

The value of i = 0, 1, . . ., m – 1. So we start from i = 0, and increase this until we get one free space. So initially when i = 0, then the h(x, i) is same as h′(x).

CLOSED HASHING

Quadratic Probing -- Example

- Example:
  - Table Size is 11 (0..10)
  - Hash Function: **h(x) = x mod 11**
  - Insert keys: 20, 30, 2, 13, 25, 24, 10, 9
    - 20 mod 11 = 9
    - 30 mod 11 = 8
    - 2 mod 11 = 2
    - 13 mod 11 = 2 → $2+1^2=3$
    - 25 mod 11 = 3 → $3+1^2=4$
    - 24 mod 11 = 2 → $2+1^2$, $2+2^2=6$
    - 10 mod 11 = 10
    - 9 mod 11 = 9 → $9+1^2$, $9+2^2$ mod 11, $9+3^2$ mod 11 =7

0	
1	
2	2
3	13
4	25
5	
6	**24**
7	**9**
8	30
9	20
10	10

CLOSED HASHING

Let us consider a simple hash function as “key mod 7” and sequence of keys as 50, 700, 76, 85, 92, 73, 101

If the slot hash(x) % S is full, then we try (hash(x) + 1*1) % S.

If (hash(x) + 1*1) % S is also full, then we try (hash(x) + 2*2) % S.

If (hash(x) + 2*2) % S is also full, then we try (hash(x) + 3*3) % S.

This process is repeated for all the values of i until an empty slot is found.

QUADRATIC PROBING

Quadratic Probing Example

0	
1	
2	
3	
4	
5	
6	

Initial Empty Table

0	
1	50
2	
3	
4	
5	
6	

Insert 50

0	700
1	50
2	
3	
4	
5	
6	76

Insert 700 and 76

0	700
1	50
2	85
3	
4	
5	
6	76

Insert 85:
Collision occurs.
Insert at 1 + 1*1 position

0	700
1	50
2	85
3	73
4	101
5	92
6	76

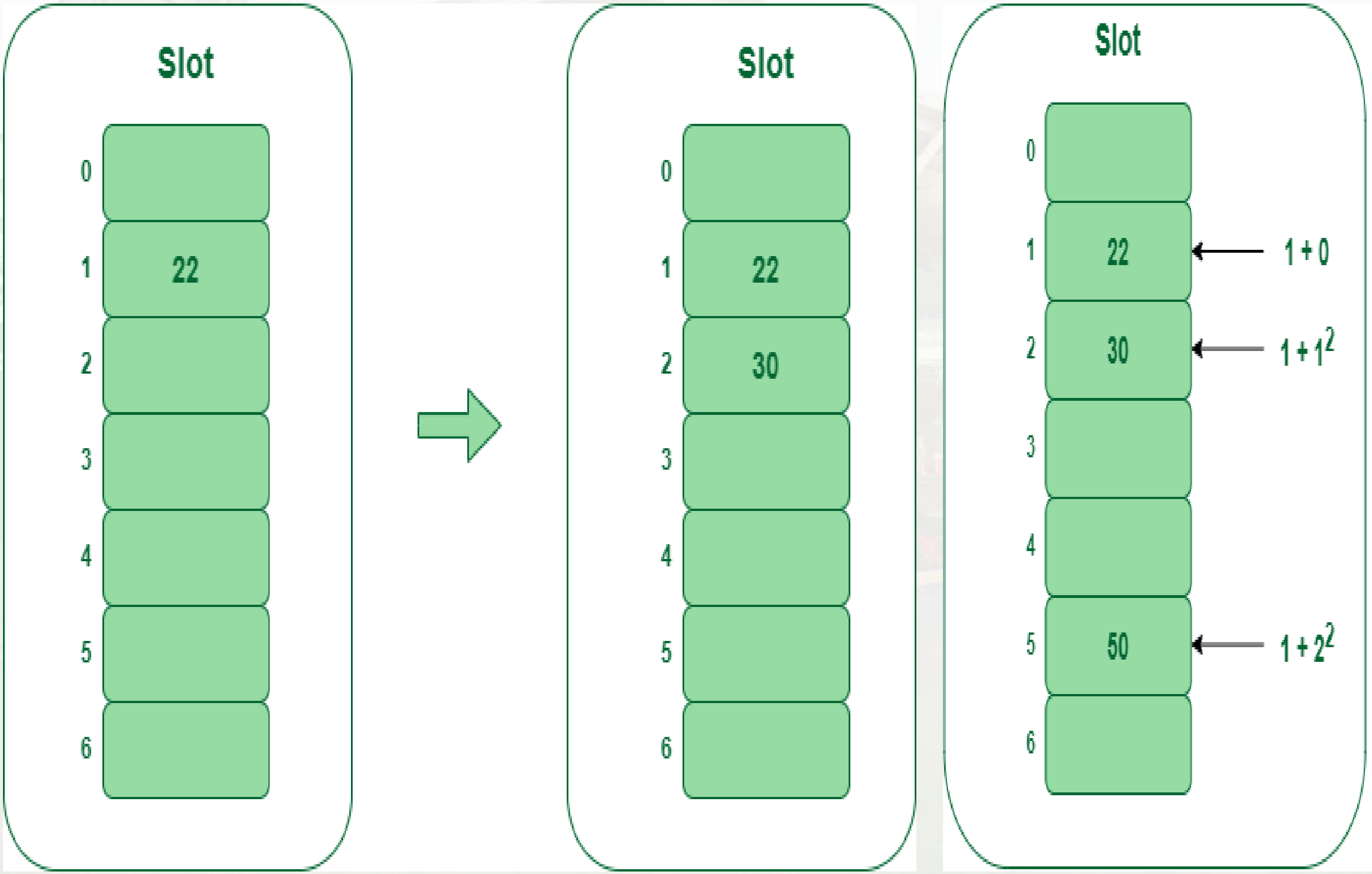
Insert 73 and 101

0	700
1	50
2	85
3	
4	
5	92
6	76

Insert 92:
Collision occurs at 1.
Collision occurs at 1 + 1*1 position
Insert at 1 + 2*2 position.

QUADRATIC PROBING

Example: Let us consider table Size = 7, hash function as Hash(x) = x % 7 and collision resolution strategy to be f(i) = i^2 . Insert = 22, 30, and 50.



QUADRATIC PROBING

Using **Linear probing** and Quadratic probing, insert the following values in the hash table of size 10.Show how many collisions occur in each iterations 28, 55, 71, 67, 11, 10, 90, 44 (Linear Probing)

insert (28)	
0	
1	
2	
3	
4	
5	
6	
7	
8	28
9	

insert (55,)	
0	
1	
2	
3	
4	
5	55,
6	
7	
8	28
9	

insert (71)	
0	
1	71
2	
3	
4	
5	55,
6	
7	
8	28
9	

insert (67)	
0	
1	71
2	
3	
4	
5	55,
6	
7	67
8	28
9	

insert (11)	
0	
1	71
2	11
3	
4	
5	55,
6	
7	67
8	28
9	

Collision

insert (10)	
0	10
1	71
2	11
3	
4	
5	55,
6	
7	67
8	28
9	

insert (90)	
0	10
1	71
2	11
3	90,
4	
5	55,
6	
7	67
8	28
9	

Collision

insert (44)	
0	10
1	71
2	11
3	90,
4	44
5	55,
6	
7	67
8	28
9	

QUADRATIC PROBING

Using Linear probing and **Quadratic probing**, insert the following values in the hash table of size 10.Show how many collisions occur in each iterations 28, 55, 71, 67, 11, 10, 90, 44

insert (28)	
0	
1	
2	
3	
4	
5	
6	
7	
8	28
9	

insert (55,)	
0	
1	
2	
3	
4	
5	55,
6	
7	
8	28
9	

insert (71)	
0	
1	71
2	
3	
4	
5	55,
6	
7	
8	28
9	

insert (67)	
0	
1	71
2	
3	
4	
5	55,
6	
7	67
8	28
9	

insert (11)	
0	
1	71
2	11
3	
4	
5	55,
6	
7	67
8	28
9	

11%10 = 1 *Collision*
(11+1)%10 = 2

insert (10)	
0	10
1	71
2	11
3	
4	
5	55,
6	
7	67
8	28
9	

insert (90,)	
0	10
1	71
2	11
3	
4	90,
5	55,
6	
7	67
8	28
9	

90%10 = 0 *Collision*
(90+1)%10 = 1 *Collision*
(90+4)%10 = 4

insert (44)	
0	10
1	71
2	11
3	44
4	90,
5	55,
6	
7	67
8	28
9	

44%10 = 4 *Collision*
(44+1)%10 = 5 *Collision*
(44+4)%10 = 8 *Collision*
(44+9)%10 = 3

Open Addressing Techniques

3. Double Hashing-

- We use another hash function hash2(x) and look for i * hash2(x) bucket in ith iteration.
- It requires more computation time as two hash functions need to be computed.
- Double hashing is a collision resolving technique in Open Addressed Hash tables. Double hashing uses the idea of applying a second hash function to key when a collision occurs.

DOUBLE HASHING

Double hashing : It is a collision resolving technique in Open Addressed Hash tables. Double hashing uses the idea of applying a second hash function to key when a collision occurs.

Advantages of Double hashing

1. The advantage of Double hashing is that it is one of the best form of probing, producing a uniform distribution of records throughout a hash table.
2. This technique does not yield any clusters.
3. It is one of effective method for resolving collisions

CLOSED HASHING

Double hashing : A good second Hash function is:

1. It must never evaluate to zero
2. Must make sure that all cells can be probed

Lets say, Hash1 (key) = key % 13

Hash2 (key) = 7 – (key % 7)

Hash1(19) = 19 % 13 = 6

Hash1(27) = 27 % 13 = 1

Hash1(36) = 36 % 13 = 10

Hash1(10) = 10 % 13 = 10

Hash2(10) = 7 – (10%7) = 4

(Hash1(10) + 1*Hash2(10))%13= 1

(Hash1(10) + 2*Hash2(10))%13= 5

Collision

CLOSED HASHING

Double hashing :

Double hashing can be done using :

(hash1(key) + i * hash2(key)) % TABLE_SIZE

Here hash1() and hash2() are hash functions and TABLE_SIZE is size of hash table.

(We repeat by increasing i when collision occurs)

First hash function is typically hash1(key) = key % TABLE_SIZE

A popular second hash function is : hash2(key) = PRIME – (key % PRIME) where PRIME is a prime smaller than the TABLE_SIZE.

CLOSED HASHING

The idea behind double hashing is fairly simple,

1. Take the key you want to store on the hash-table.

2. Apply the first hash function h_1 (key) over your key to get the location to store the key.

3. If the location is empty, place the key on that location.

4. If the location is already filled, apply the second hash function h_2 *(key)* in combination with the first hash function h_1 *(key)* to get the new location for the key.

CLOSED HASHING

Suppose, we have a hash table of size **11**. We want to insert keys **20, 34, 45, 70, 56** in the hash table. Let's insert the keys into hash table using the following double hash functions:

$h_1(k) = k \bmod 11$ (first hash function)

$h_2(k) = 8 - (k \bmod 8)$ (second hash function)

first, we will create a hash table of size 11.

CLOSED HASHING

Steps	Key	Hash Function	Index	Description
1	20	$h_1(20) = 20 \bmod 11$	9	No collision occurs.
2	34	$h_1(34) = 34 \bmod 11$	1	No collision occurs.
3	45	$h_1(45) = 45 \bmod 11$	**1**	Collision occur because index 1 is already occupied by 34. Now we will use the second hash function to calculate the index for the key 45.
		$h_2(45) = 8 - (45 \bmod 8) = 3$ $h(45, 1) = (1 + 1 * 3) \bmod 11$	4	Here, we have taken the value if i 1 because first collision occurs. Note that here h_2 (k) and 11 are relatively prime. The value of h_2 (k) must be less than the table size.
4	70	$h_1(70) = 70 \bmod 11$	**4**	Collision occur because index 4 is already occupied by 45. Now we will use the second hash function to calculate the index for the key 70.

CLOSED HASHING

		$h_2(70) = 8 - (70 \mod 8) = 2$ $h(70, 1) = (4 + 1 * 2) \mod 11$	6	Here, we have taken the value if i 1 because first collision occurs. Note that here h_2 (k) and 11 are relatively prime. The value of h_2 (k) must be less than the table size.
5	56	$h_1(56) = 56 \mod 11$	**1**	Collision occur because index 1 is already occupied by 34. Now we will use the second hash function to calculate the index for the key 56.
		$h_2(56) = 8 - (56 \mod 8) = 8$ $h(56, 1) = (1 + 1 * 8) \mod 11$	**9**	Again, collision occur. The index 9 is already occupied by 20.
		$h(56, 2) = (1 + 2 * 8) \mod 11$	**6**	Here, the value of i is incremented by 1 because collision occurs second times. We note that again collision occurs because the index 6 is already occupied by 70.
		$h(56, 3) = (1 + 3 * 8) \mod 11$	3	Here, the value of i is incremented by 1. The third index is empty. So, we will store 56 at index 3.

CLOSED HASHING

After inserting all the keys, the hash table looks like the following.

0	
1	34
2	
3	56
4	45
5	
6	70
7	
8	
9	20
10	

DOUBLE HASHING EXAMPLE

Double Hashing Example :

Imagine you need to store some items inside a hash table of size 20. The values given are: (16, 8, 63, 9, 27, 37, 48, 5, 69, 34, 1).

h1(n)=n%20

h2(n)=n%13

n h(n, i) = (h1 (n) + ih2(n)) mod 20

Open Vs Close Hashing

Separate Chaining	Open Addressing
Keys are stored inside the hash table as well as outside the hash table.	All the keys are stored only inside the hash table.
The number of keys to be stored in the hash table can even exceed the size of the hash table.	The number of keys to be stored in the hash table can never exceed the size of the hash table.
Deletion is easier.	Deletion is difficult.
Extra space is required for the pointers to store the keys outside the hash table.	No extra space is required.
Cache performance is poor. This is because of linked lists which store the keys outside the hash table.	Cache performance is better. This is because here no linked lists are used.
Some buckets of the hash table are never used which leads to wastage of space.	Buckets may be used even if no key maps to those particular buckets.

Rehashing

Rehashing is a collision resolution technique.

Rehashing is a technique in which the table is resized, i.e., the size of table is doubled by creating a new table. It is preferable is the total size of table is a prime number. There are situations in which the rehashing is required.

- When table is completely full
- With quadratic probing when the table is filled half.
- When insertions fail due to overflow.

In such situations, we have to transfer entries from old table to the new table by re computing their positions using hash functions

Rehashing

Hash Table with input 7, 16, 33, 42

h(x) = x mod 7

0	7
1	
2	16
3	
4	
5	33
6	42

Rehashing

h(x) = x mod 17

0	
1	
2	
3	
4	
5	22
6	
7	7
8	42
9	
10	
11	
12	
13	
14	
15	33
16	16

Insert 22

0	7
1	22
2	16
3	
4	
5	33
6	42

Rehashing

As the name suggests, rehashing means hashing again. Basically, when the load factor increases to more than its pre-defined value (default value of load factor is 0.75), the complexity increases. So to overcome this, the size of the array is increased (doubled) and all the values are hashed again and stored in the new double sized array to maintain a low load factor and low complexity.

Rehashing

Why rehashing?

Rehashing is done because whenever key value pairs are inserted into the map, the load factor increases, which implies that the time complexity also increases as explained above. This might not give the required time complexity of O(1).

Hence, rehash must be done, increasing the size of the bucket Array so as to reduce the load factor and the time complexity

Rehashing

How Rehashing is done?

Rehashing can be done as follows:

- For each addition of a new entry to the map, check the load factor.
- If it's greater than its pre-defined value (or default value of 0.75 if not given), then Rehash.
- For Rehash, make a new array of double the previous size and make it the new bucket array.
- Then traverse to each element in the old bucket Array and call the insert() for each so as to insert it into the new larger bucket array.

ISSUES IN HASHING

- **Hash Collisions:** Hashing can produce the same hash value for different keys, leading to hash collisions. To handle collisions, we need to use collision resolution techniques like chaining or open addressing.

- **Hash Function Quality:** The quality of the hash function determines the efficiency of the hashing algorithm. A poor-quality hash function can lead to more collisions, reducing the performance of the hashing algorithm**.**

Extensible/Extendible Hashing

1. The dynamic hashing method is used to overcome the problems of static hashing like bucket overflow.
2. In this method, data buckets grow or shrink as the records increases or decreases. This method is also known as **Extendable hashing method**.
3. This method makes hashing dynamic, i.e., it allows insertion or deletion without resulting in poor performance.

Extensible/Extendible Hashing

- **How to search a key**

1. First, calculate the hash address of the key.
2. Check how many bits are used in the directory, and these bits are called as i.
3. Take the least significant i bits of the hash address. This gives an index of the directory.
4. Now using the index, go to the directory and find bucket address where the record might be.

Extensible/Extendible Hashing

- **How to insert a new record**

1. Firstly, you have to follow the same procedure for retrieval, ending up in some bucket.
2. If there is still space in that bucket, then place the record in it.
3. If the bucket is full, then we will split the bucket and redistribute the records.

Extensible/Extendible Hashing

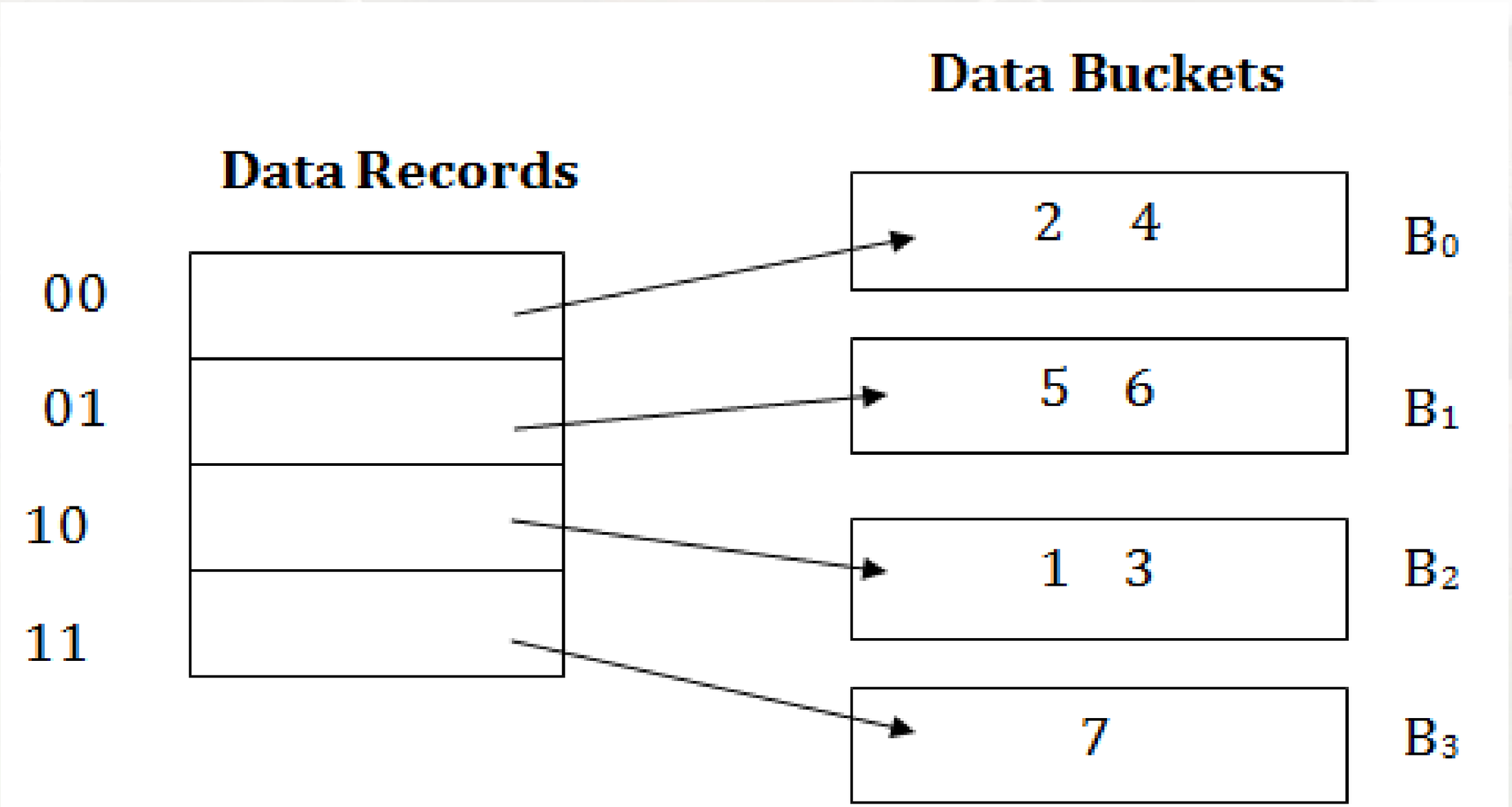
- **For example :**

Consider the following grouping of keys into buckets, depending on the prefix of their hash address:

Key	Hash address
1	11010
2	00000
3	11110
4	00000
5	01001
6	10101
7	10111

Extensible/Extendible Hashing

The last two bits of 2 and 4 are 00. So it will go into bucket B0. The last two bits of 5 and 6 are 01, so it will go into bucket B1. The last two bits of 1 and 3 are 10, so it will go into bucket B2. The last two bits of 7 are 11, so it will go into B3.

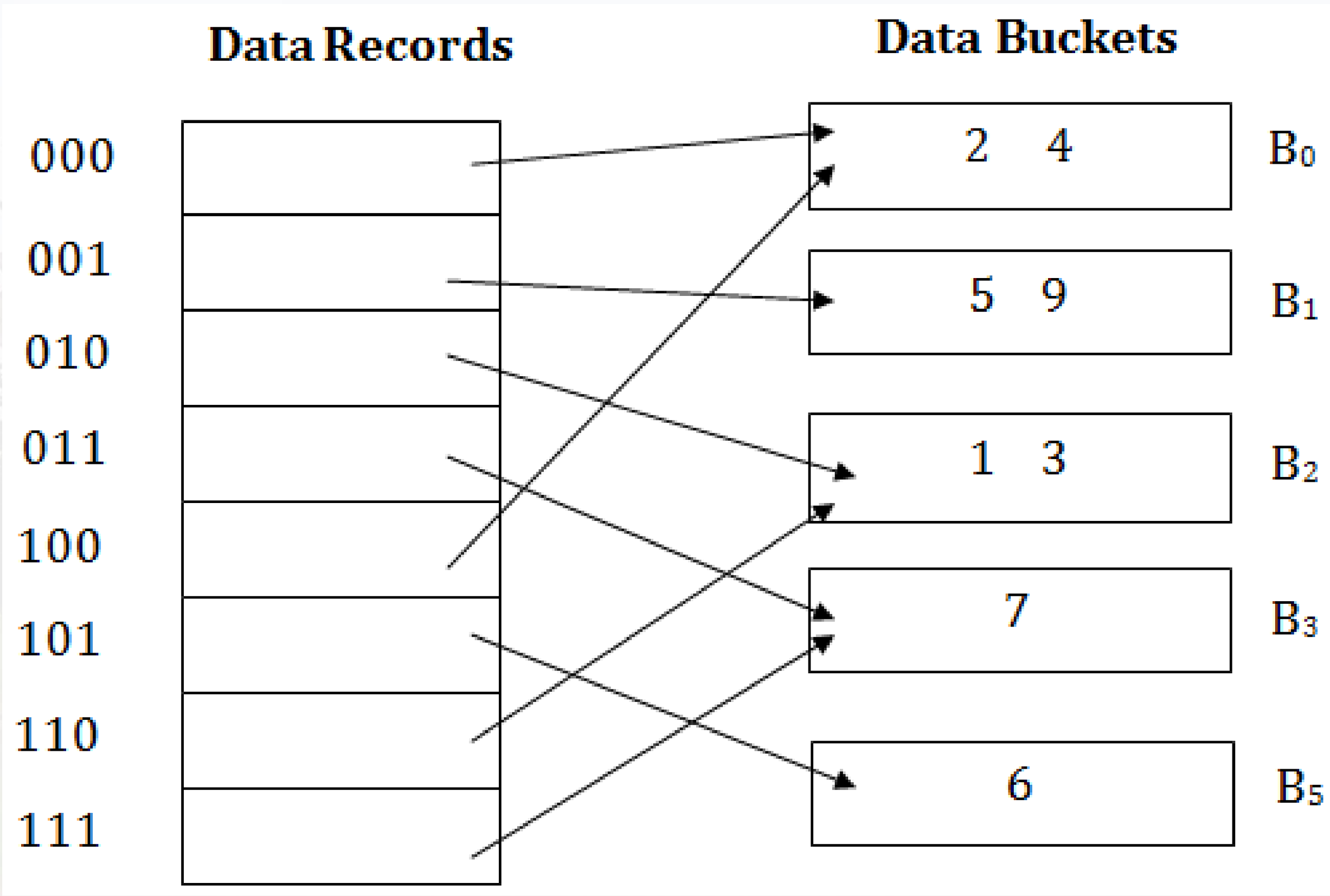


Extensible/Extendible Hashing

Insert key 9 with hash address 10001 into the above structure:

1. Since key 9 has hash address 10001, it must go into the first bucket. But bucket B1 is full, so it will get split.
2. The splitting will separate 5, 9 from 6 since last three bits of 5, 9 are 001, so it will go into bucket B1, and the last three bits of 6 are 101, so it will go into bucket B5.
3. Keys 2 and 4 are still in B0. The record in B0 pointed by the 000 and 100 entry because last two bits of both the entry are 00.
4. Keys 1 and 3 are still in B2. The record in B2 pointed by the 010 and 110 entry because last two bits of both the entry are 10.
5. Key 7 are still in B3. The record in B3 pointed by the 111 and 011 entry because last two bits of both the entry are 11.

Extensible/Extendible Hashing



Advantages of Extensible Hashing

1. In this method, the performance does not decrease as the data grows in the system. It simply increases the size of memory to accommodate the data.
2. In this method, memory is well utilized as it grows and shrinks with the data. There will not be any unused memory lying.
3. This method is good for the dynamic database where data grows and shrinks frequently

Disadvantages of Extensible Hashing

1. In this method, if the data size increases then the bucket size is also increased.
2. In this case, the bucket overflow situation will also occur. But it might take little time to reach this situation than static hashing.

Example of Extensible Hashing

Example based on Extendible Hashing: Now, let us consider a prominent example of hashing the following elements: 16,4,6,22,24,10,31,7,9,20,26.

Bucket Size: 3 (Assume)

Link : https://www.geeksforgeeks.org/extendible-hashing-dynamic-approach-to-dbms/

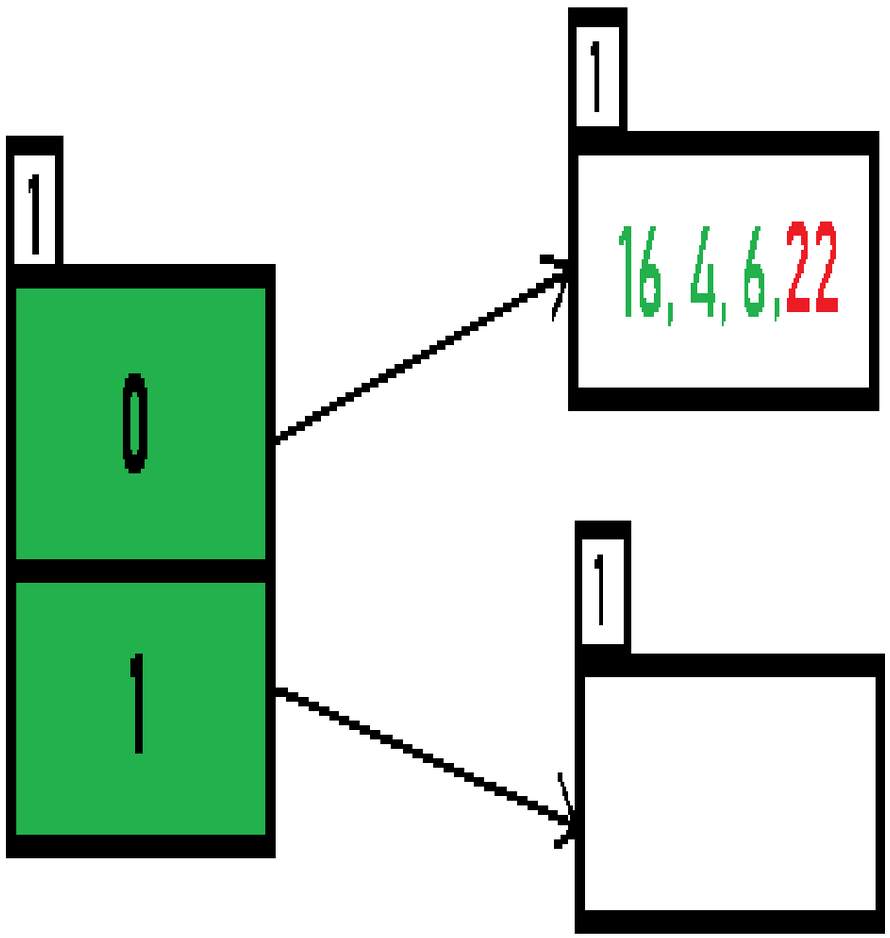
Example of Extensible Hashing

Sr. No.	Decimal	Binary
1	16	10000
2	4	00100
3	6	00110
4	22	10110
5	24	11000
6	10	01010
7	31	11111
8	7	00111
9	9	01001
10	20	10100
11	26	11010

Example of Extensible Hashing

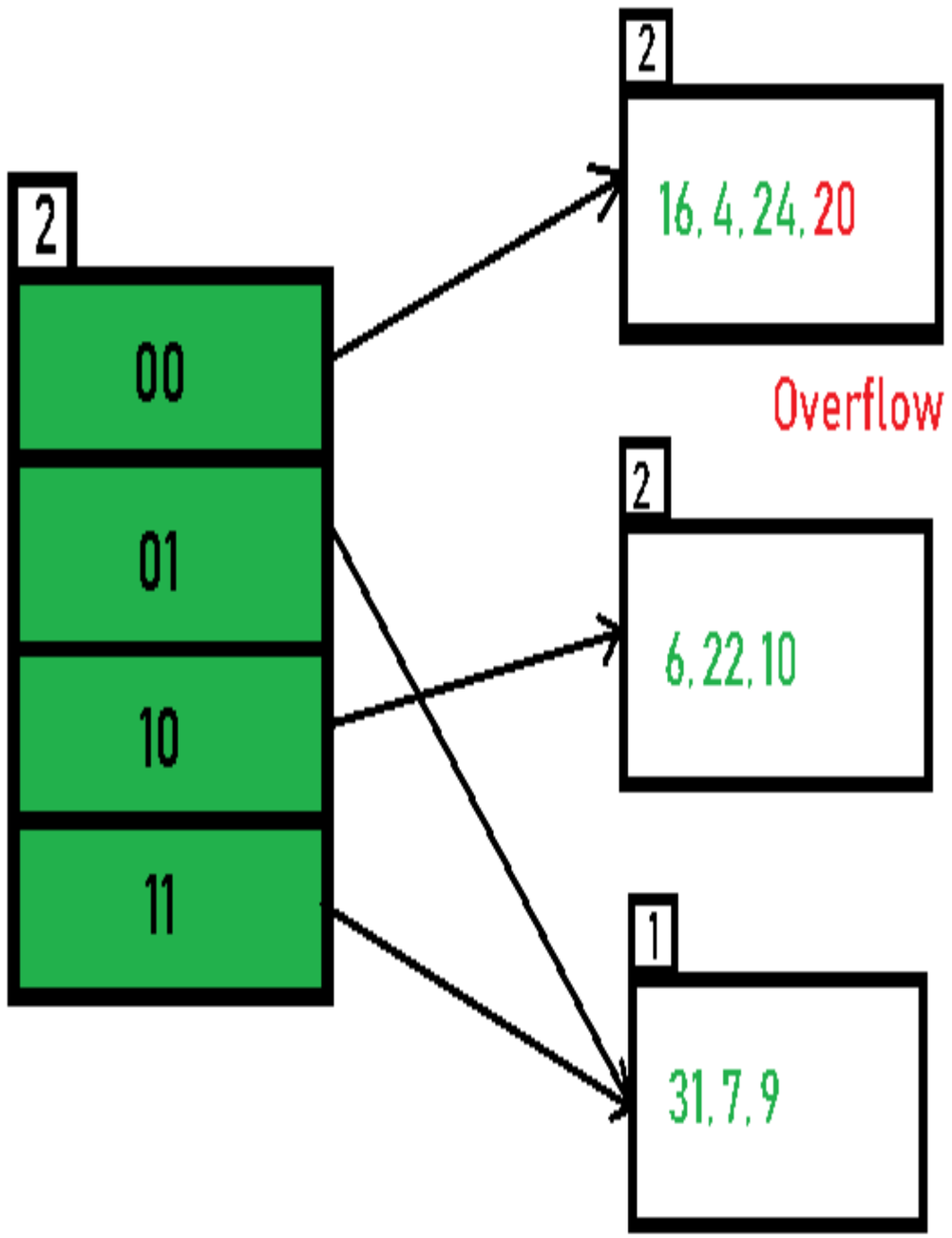
OverFlow Condition

Here, Local Depth=Global Depth



Hash(22)=10110

OverFlow, Local Depth= Global Depth



Hash(20)=10100

Example of Extensible Hashing

Solution: First, calculate the binary forms of each of the given numbers.

16 - 10000

4 - 00100

6 - 00110

22 - 10110

24 - 11000

10 - 01010

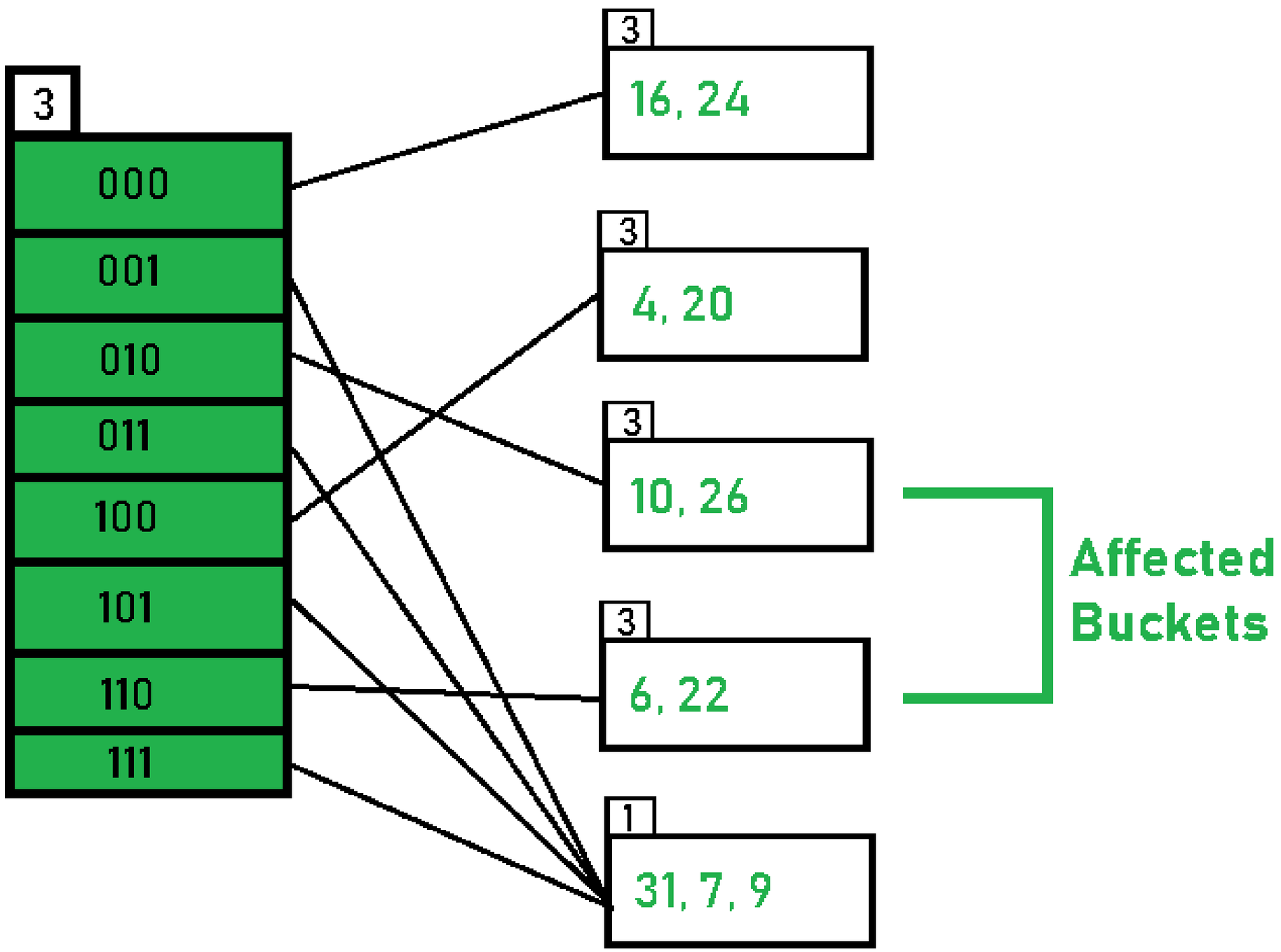
31 - 11111

7 - 00111

9 - 01001

20 - 10100

26 - 11010



References

1. https://www.javatpoint.com/double-hashing-in-java
2. https://www.scaler.com/topics/quadratic-probing/
3. https://www.geeksforgeeks.org/quadratic-probing-in-hashing/
4. https://www.bucketstudy.com/2021/09/DSA01.html
5. https://www.upgrad.com/blog/hashing-in-data-structure/
6. https://quescol.com/data-structure/linear-probing
7. https://prac-code.blogspot.com/2013/06/chaining-without-replacement.html
8. https://www.geeksforgeeks.org/open-addressing-collision-handling-technique-in-hashing/

References

9. https://www.ques10.com/p/32048/using-linear-probing-and-quadratic-probing-insert-/

10. https://dev.to/aws-builders/intro-into-hashing-3nf6

https://www.geeksforgeeks.org/extendible-hashing-dynamic-approach-to-dbms/

THANK YOU!!!

My Blog : https://anandgharu.wordpress.com/

Email : gharu.anand@gmail.com